

Lembar Jawaban Tugas Besar

EB3103 – Pengolahan Sinyal Biomedis

Semester 1 – 2025/2026

Michael Liebing	Aliya Shafa Imani	Filipus Putra Atmadja	Ananda Puteri Gitasya	Nurbaedah Awaliyah
18323016	18321009	18323008	18323007	18322027
				
1. Membuat kode Matlab soal 1a, 1b, 1c, 1d, 2a, 2b, 3a, 3b, dan 3c 2. Memasukkan gambar-gambar hasil menjalani kode Matlab soal 1a, 1b, 1c, 1d, 2a, 2b, 3a, 3b, dan 3c. 3. Membuat analisis hasil menjalani kode Matlab soal 1a, 2a, dan 2b 4. Memberi saran terkait analisis hasil menjalani kode Matlab soal 1b, 1c, 1d, 3a, 3b, dan 3c. 5. Merapikan dokumen lembar jawaban tugas besar. 6. Membuat GitHub sebagai wadah koordinasi pembuatan kode Matlab.	1. Membuat kode Matlab soal 1b dan 3a 2. Memasukkan gambar hasil menjalani kode Matlab soal 1b dan 3a 3. Membuat analisis hasil menjalani kode Matlab soal 1b dan 3a 4. Memberi saran terkait analisis hasil menjalani kode Matlab 3b dan 3c	1. Membuat kode Matlab soal 1c 2. Membuat analisis hasil Matlab soal 1c 3. Memberikan saran dan masukan untuk perancangan soal 1d	1. Membuat kode matlab 1d 2. Menginterpretasikan pole-zero untuk kode matlab 1c 3. Memberikan masukan untuk alternatif kemungkinan filter yang digunakan pada kode matlab soal 1 4. Membuat analisis tahapan kode Matlab 1d dan sesuai dengan materi yang digunakan 5. Memberikan saran dan masukan analisis 1c, 3b, dan 3c	1. Membuat kode matlab 1c 2. Memberikan masukan filter untuk perancangan pada soal 1d 3. Memberikan saran dan masukan analisis pada soal 1c 4. Membuat analisis hasil menjalani kode no 3b dan 3c

Daftar Isi

Daftar Isi	2
Soal 1: Audio	3
Soal 2: ECG	30
Soal 3: Real-time processing	36
Referensi	43

Nilai: 10

kerapihan
10

Soal 1: Audio

Diberikan sinyal audio hasil perekaman di dekat palang perlintasan kereta api:

Nama file	Parameter pengukuran
kereta.aac	- 1 buah sinyal audio format *.aac dalam arbitrary unit; - ADC: 44100 sampel/detik; - Stereo;

Anda diminta untuk membuat program Matlab yang dapat:

a. membaca dan menampilkan sinyal domain waktu “teeet..tooot..teeet..tooot..teeet..tooot.. ...”.

Tips: hitung durasi “teeet..” sebelum berganti menjadi “tooot” dan sebaliknya? apakah komponen sinyal tersebut bisa dikategorikan sinyal periodik? jika bisa, berapa perioda sinyal “teeet..” dan perioda sinyal “tooot..”?

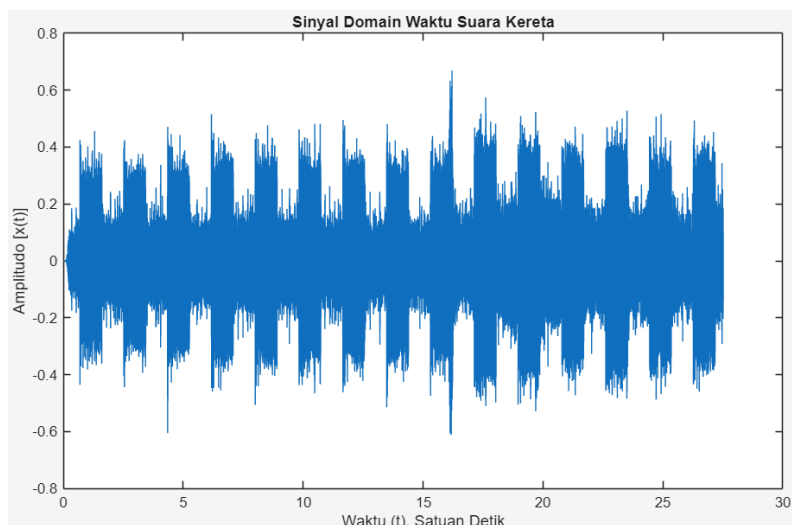
Jawab:

Analisis sinyal domain waktu dapat dilaksanakan dengan bantuan kode Matlab [3] dan jika kode tersebut dijalankan, teks berikut akan muncul dalam *command window*,

Audio speaker kanan dan kiri sama

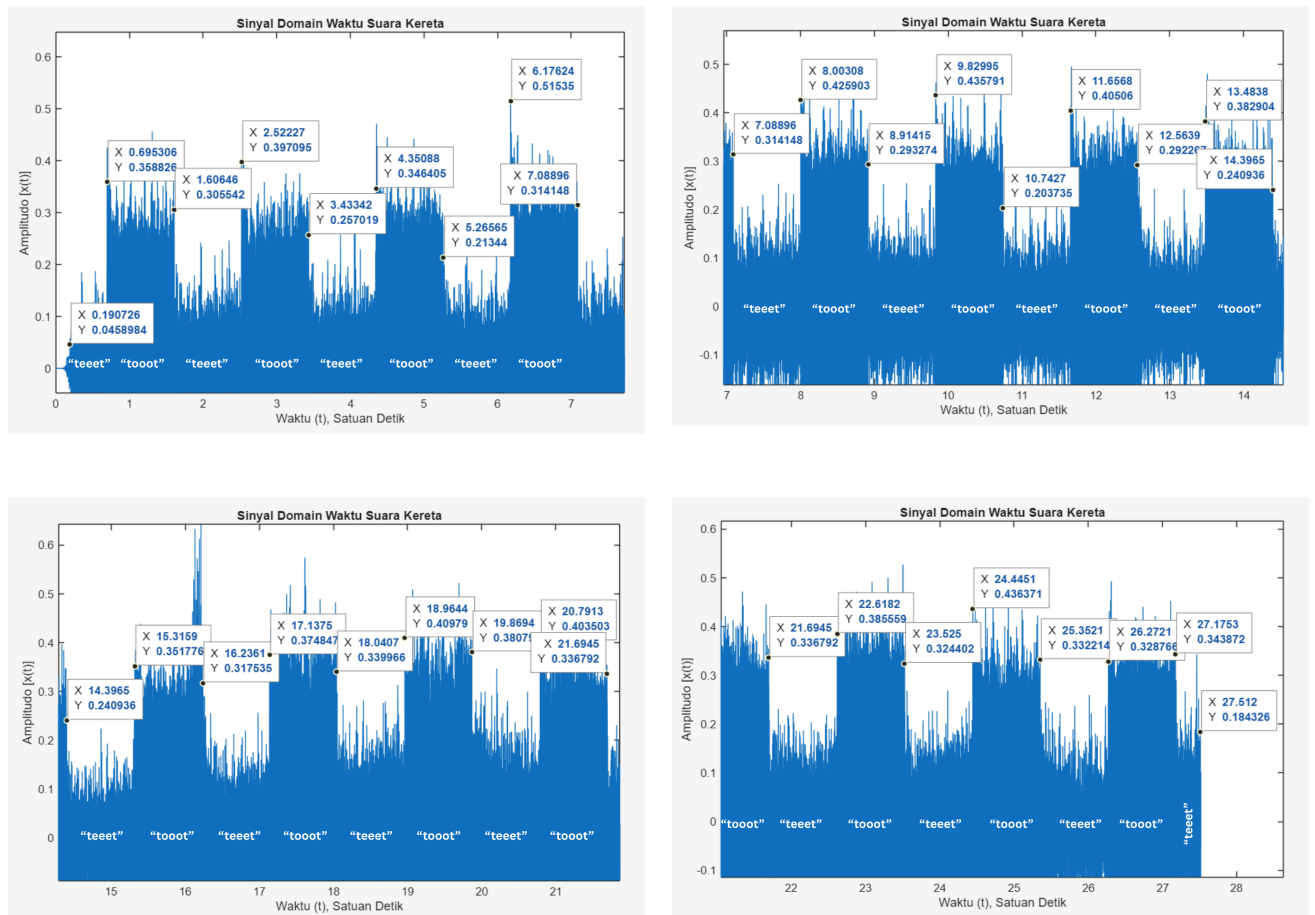
Gambar 1 - Tampilan Command Window Program Soal 1a

Hal tersebut menunjukkan bahwa audio pada *speaker* kanan dan kiri memiliki nilai amplitudo yang sama pada setiap sampelnya sehingga sinyal domain waktu yang ditampilkan dapat menggunakan audio dari salah satu *speaker* saja. Berikut adalah hasil tampilan sinyal domain waktu dari file “kereta.aac”,



Gambar 2 - Sinyal Domain Waktu File “kereta.aac”

Demi menghitung durasi dari suara “teeet” dan “tooot”, penentuan waktu awal dan akhir setiap kemunculan suara tersebut dapat dilakukan secara manual berdasarkan sinyal domain waktu yang ditampilkan pada Gambar 3. Berikut waktu awal dan akhir dari setiap kemunculan suara “teeet” dan “tooot” pada sinyal domain waktu file “kereta.aac”,



Gambar 3 - Pengukuran Durasi Suara "teeet" dan "tooot"

Berikut perhitungan durasi dari setiap suara "teeet" dan "tooot" pada sinyal domain waktu "kereta.aac",

$$\begin{aligned}
 t_{teeet1} &= 0,695306 - 0,190726 = 0,50458 \text{ s} & t_{tooot1} &= 1,60646 - 0,695306 = 0,911154 \text{ s} \\
 t_{teeet2} &= 2,52227 - 1,60646 = 0,91581 \text{ s} & t_{tooot2} &= 3,43342 - 2,52227 = 0,91115 \text{ s} \\
 t_{teeet3} &= 4,35088 - 3,43342 = 0,91746 \text{ s} & t_{tooot3} &= 5,26565 - 4,35088 = 0,91477 \text{ s} \\
 t_{teeet4} &= 6,17624 - 5,26565 = 0,91059 \text{ s} & t_{tooot4} &= 7,08896 - 6,17624 = 0,91272 \text{ s} \\
 t_{teeet5} &= 8,00308 - 7,08896 = 0,91412 \text{ s} & t_{tooot5} &= 8,91415 - 8,00308 = 0,91107 \text{ s} \\
 t_{teeet6} &= 9,82995 - 8,91415 = 0,9158 \text{ s} & t_{tooot6} &= 10,7427 - 9,82995 = 0,91275 \text{ s} \\
 t_{teeet7} &= 11,6568 - 10,7427 = 0,9141 \text{ s} & t_{tooot7} &= 12,5639 - 11,6568 = 0,9071 \text{ s} \\
 t_{teeet8} &= 13,4838 - 12,5639 = 0,9199 \text{ s} & t_{tooot8} &= 14,3965 - 13,4838 = 0,9127 \text{ s} \\
 t_{teeet9} &= 15,3159 - 14,3965 = 0,9194 \text{ s} & t_{tooot9} &= 16,2361 - 15,3159 = 0,9202 \text{ s} \\
 t_{teeet10} &= 17,1375 - 16,2361 = 0,9014 \text{ s} & t_{tooot10} &= 18,0407 - 17,1375 = 0,9032 \text{ s} \\
 t_{teeet11} &= 18,9644 - 18,0407 = 0,9237 \text{ s} & t_{tooot11} &= 19,8694 - 18,9644 = 0,905 \text{ s} \\
 t_{teeet12} &= 20,7913 - 19,8694 = 0,9219 \text{ s} & t_{tooot12} &= 21,6945 - 20,7913 = 0,9032 \text{ s} \\
 t_{teeet13} &= 22,6182 - 21,6945 = 0,9237 \text{ s} & t_{tooot13} &= 23,525 - 22,6182 = 0,9068 \text{ s} \\
 t_{teeet14} &= 24,4451 - 23,525 = 0,9201 \text{ s} & t_{tooot14} &= 25,3521 - 24,4451 = 0,907 \text{ s}
 \end{aligned}$$

$$t_{teeet15} = 26,2721 - 25,3521 = 0,92 \text{ s}$$

$$t_{teeet16} = 27,512 - 27,1753 = 0,3367 \text{ s}$$

$$t_{tooot15} = 27,1753 - 26,2721 = 0,9032 \text{ s}$$

Berdasarkan hasil perhitungan tersebut, terlihat bahwa terdapat 2 nilai durasi suara “teeet” yang berbeda dengan durasi lainnya, yaitu $t_{teeet1} = 0,50458 \text{ s}$ dan $t_{teeet16} = 0,3367 \text{ s}$. Hal tersebut disebabkan oleh terpotongnya durasi suara tersebut dalam perekaman audio “kereta.aac” sehingga kedua durasi tersebut dapat diabaikan dalam analisis durasi suara “teeet” dan “tooot”. Rata-rata dari durasi suara “teeet” dan “tooot” dapat dihitung sebagai berikut,

$$t_{teeet \text{ rata-rata}} = \frac{t_{teeet \text{ total}}}{14} = \frac{12,83798}{14} \text{ s}$$

$$= 0,9169985714 \text{ s}$$

$$t_{tooot \text{ rata-rata}} = \frac{t_{tooot \text{ total}}}{15} = \frac{13,642014}{15} \text{ s}$$

$$= 0,9094676 \text{ s}$$

Selain itu, nilai durasi “teeet” dan “tooot” berada pada nilai yang serupa, yaitu berada pada rentang nilai 0,9014 - 0,9237 detik, dengan perbedaan nilai durasi antara suara tersebut dapat disebabkan oleh *noise* sinyal yang dapat membuat penentuan waktu awal atau akhir suara kurang tepat. Oleh karena itu, sinyal tersebut dapat dikategorikan sebagai sinyal periodik dengan perhitungan perioda sinyal suara “teeet” dan “tooot” sebagai berikut,

$$T_{teeet2 \rightarrow 3} = 3,43342 - 1,60646 = 1,82696 \text{ s}$$

$$T_{teeet3 \rightarrow 4} = 5,26565 - 3,43342 = 1,83223 \text{ s}$$

$$T_{teeet4 \rightarrow 5} = 7,08896 - 5,26565 = 1,82331 \text{ s}$$

$$T_{teeet5 \rightarrow 6} = 8,91415 - 7,08896 = 1,82519 \text{ s}$$

$$T_{teeet6 \rightarrow 7} = 10,7427 - 8,91415 = 1,82855 \text{ s}$$

$$T_{teeet7 \rightarrow 8} = 12,5639 - 10,7427 = 1,8212 \text{ s}$$

$$T_{teeet8 \rightarrow 9} = 14,3965 - 12,5639 = 1,8326 \text{ s}$$

$$T_{teeet9 \rightarrow 10} = 16,2361 - 14,3965 = 1,8396 \text{ s}$$

$$T_{teeet10 \rightarrow 11} = 18,0407 - 16,2361 = 1,8046 \text{ s}$$

$$T_{teeet11 \rightarrow 12} = 19,8694 - 18,0407 = 1,8287 \text{ s}$$

$$T_{teeet12 \rightarrow 13} = 21,6945 - 19,8694 = 1,8251 \text{ s}$$

$$T_{teeet13 \rightarrow 14} = 23,525 - 21,6945 = 1,8305 \text{ s}$$

$$T_{teeet14 \rightarrow 15} = 25,3521 - 23,525 = 1,8271 \text{ s}$$

$$T_{teeet15 \rightarrow 16} = 27,1753 - 25,3521 = 1,8232 \text{ s}$$

$$T_{tooot1 \rightarrow 2} = 2,52227 - 0,695306 = 1,826964 \text{ s}$$

$$T_{tooot2 \rightarrow 3} = 4,35088 - 2,52227 = 1,82861 \text{ s}$$

$$T_{tooot3 \rightarrow 4} = 6,17624 - 4,35088 = 1,82536 \text{ s}$$

$$T_{tooot4 \rightarrow 5} = 8,00308 - 6,17624 = 1,82684 \text{ s}$$

$$T_{tooot5 \rightarrow 6} = 9,82995 - 8,00308 = 1,82687 \text{ s}$$

$$T_{tooot6 \rightarrow 7} = 11,6568 - 9,82995 = 1,82685 \text{ s}$$

$$T_{tooot7 \rightarrow 8} = 13,4838 - 11,6568 = 1,827 \text{ s}$$

$$T_{tooot8 \rightarrow 9} = 15,3159 - 13,4838 = 1,8321 \text{ s}$$

$$T_{tooot9 \rightarrow 10} = 17,1375 - 15,3159 = 1,8216 \text{ s}$$

$$T_{tooot10 \rightarrow 11} = 18,9644 - 17,1375 = 1,8269 \text{ s}$$

$$T_{tooot11 \rightarrow 12} = 20,7913 - 18,9644 = 1,8269 \text{ s}$$

$$T_{tooot12 \rightarrow 13} = 22,6182 - 20,7913 = 1,8269 \text{ s}$$

$$T_{tooot13 \rightarrow 14} = 24,4451 - 22,6182 = 1,8269 \text{ s}$$

$$T_{tooot14 \rightarrow 15} = 26,2721 - 24,4451 = 1,827 \text{ s}$$

$$T_{teeet \text{ rata-rata}} = \frac{T_{teeet \text{ total}}}{14} = \frac{25,56884}{14} \text{ s}$$

$$= 1,826345714 \text{ s}$$

$$T_{tooot \text{ rata-rata}} = \frac{T_{tooot \text{ total}}}{14} = \frac{25,576794}{14} \text{ s}$$

$$= 1,826913857 \text{ s}$$

Berdasarkan hasil perhitungan tersebut, nilai perioda yang diperoleh untuk sinyal suara “teeet” adalah 1,826345714 detik dan nilai perioda untuk sinyal suara “tooot” adalah 1,826913857 detik.

b. menghitung dan menampilkan *power spectrum density* (PSD) sinyal atau *spectrogram*.

Tips: lakukan eksplorasi resolusi frekuensi dan kebocoran frekuensi, serta aplikasi teorema central limit pada estimasi spektrum, yakni:

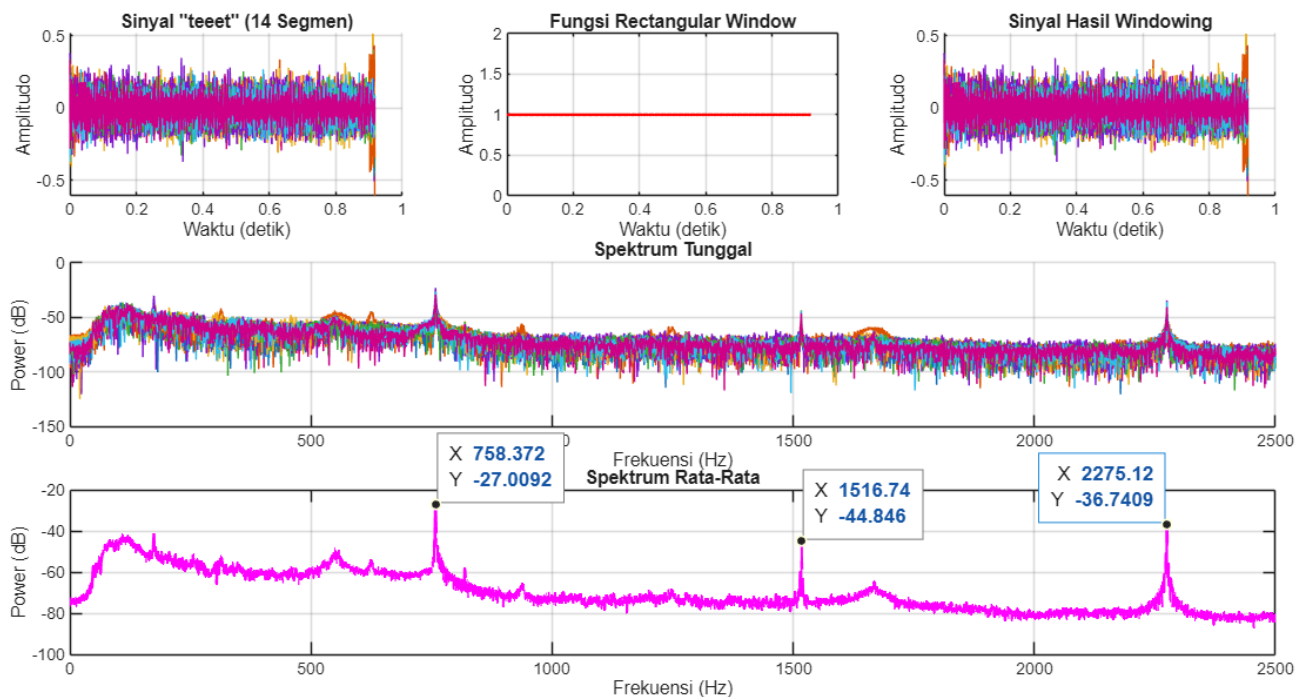
- 1) potong menjadi sekumpulan sinyal “teeet” atau sekumpulan sinyal “tooot” saja,

- 2) kalikan dengan sebuah window,
- 3) cari masing-masing spektrum, dan
- 4) rata-ratakan spektrum “teeet” dan spektrum “tooot”.

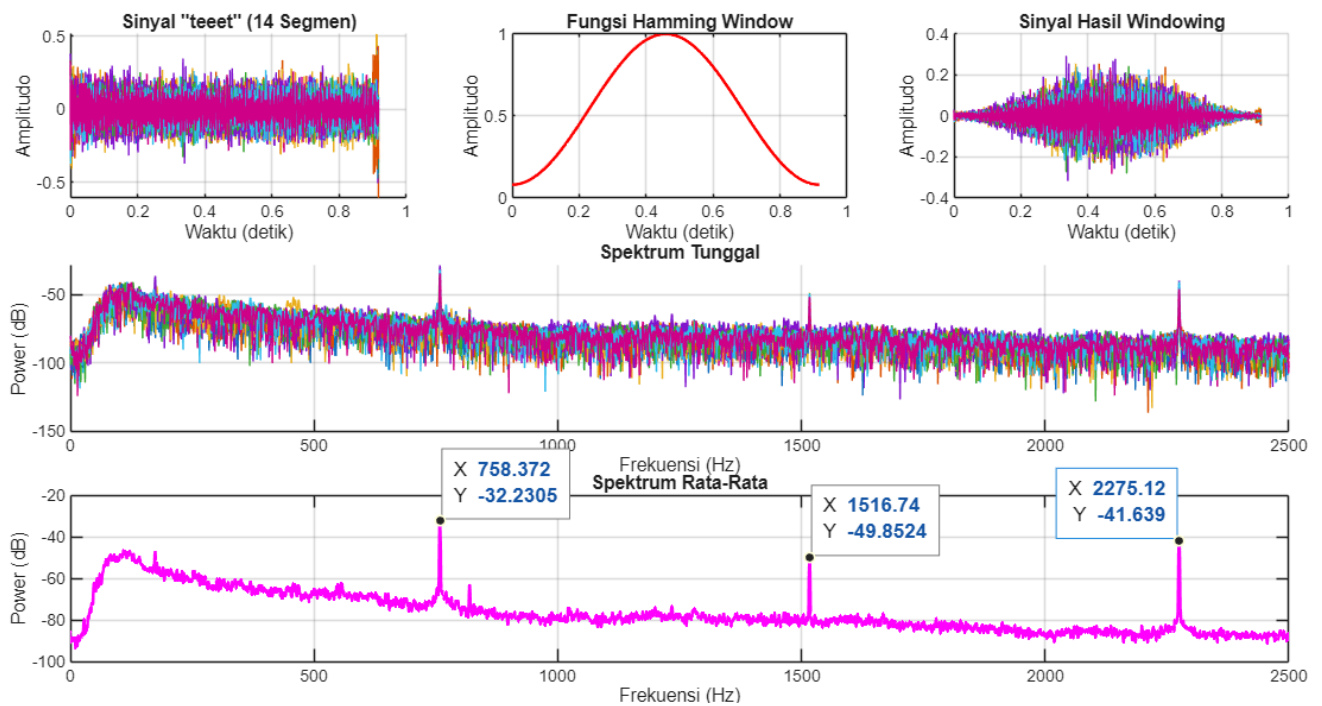
Diskusikan adakah perbedaan spektrum sinyal: “teeet..tooot..teeet..tooot..teeet..tooot.. ...” atau “tet..tot..tet..tot..tet..tot.. ...” atau “teeet” atau “tooot” atau “tet” atau “tot”. Lakukan juga eksplorasi zero padding.

Jawab:

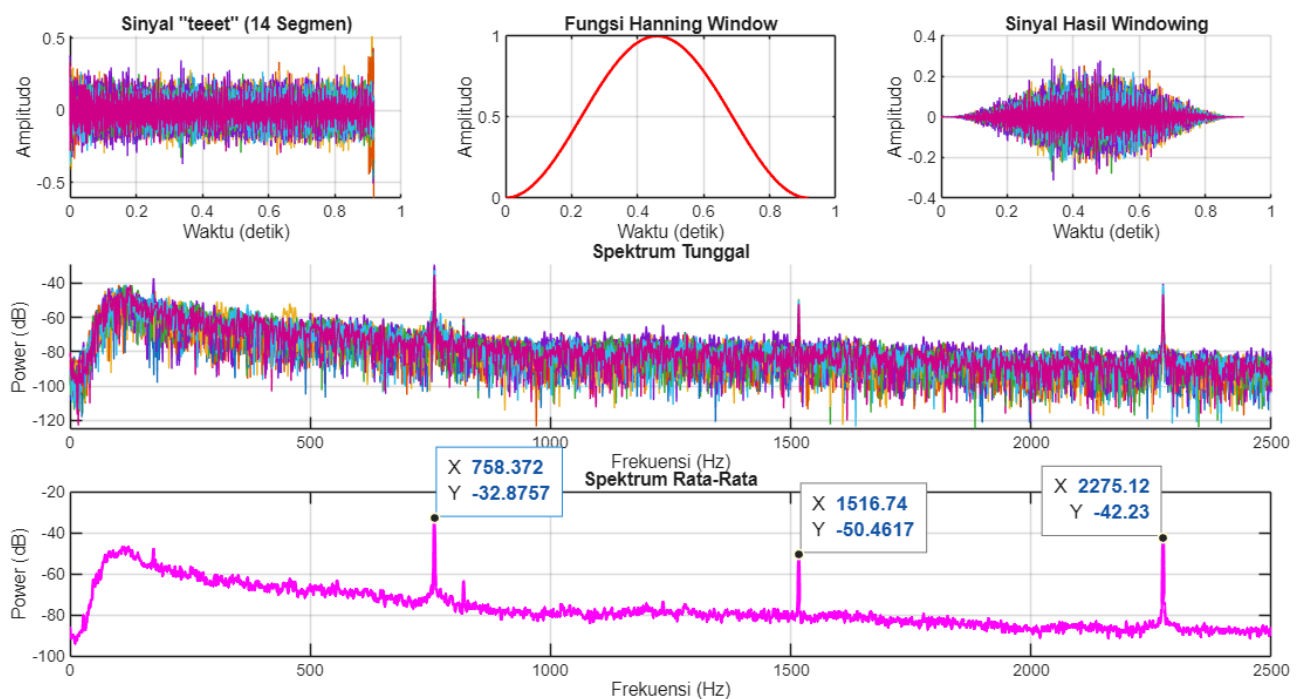
Pembuatan PSD dapat dilaksanakan dengan bantuan kode Matlab [3] dan jika kode tersebut dijalankan, PSD yang diperoleh adalah sebagai berikut,



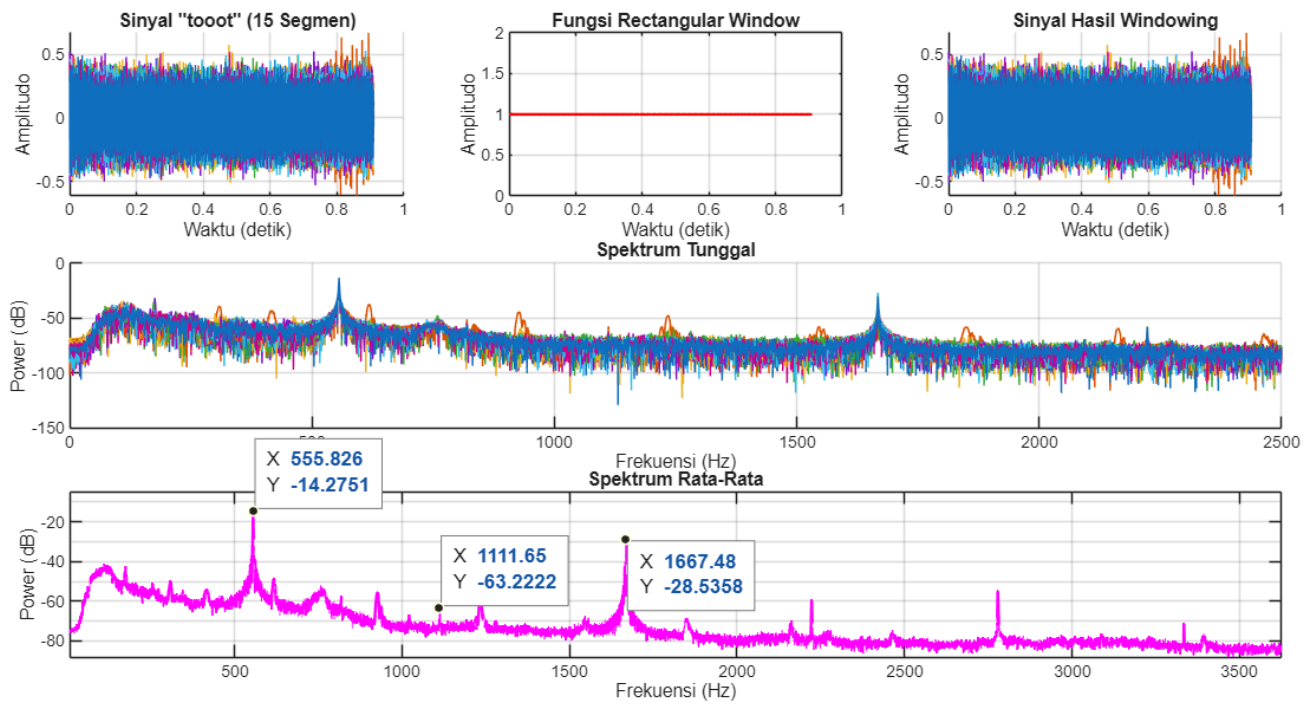
Gambar 4 - Analisis PSD Sinyal “teeet” dengan Rectangular Window



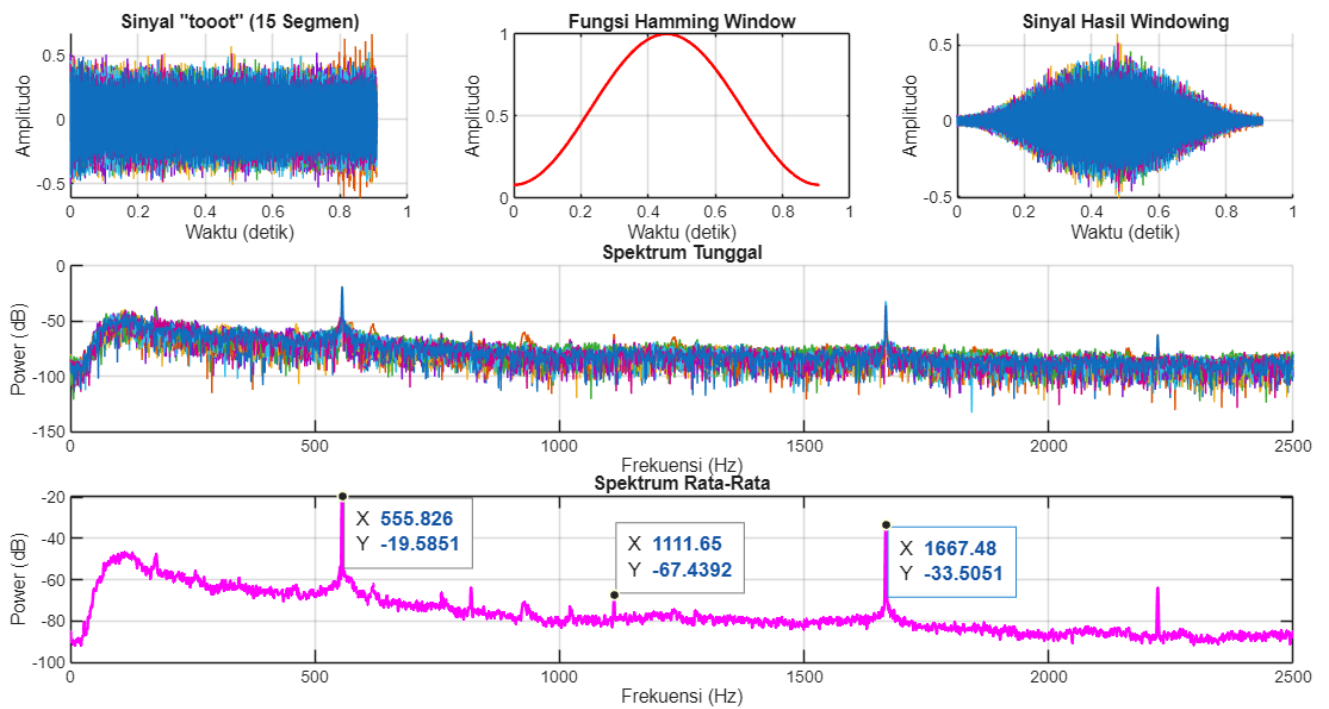
Gambar 5 - Analisis PSD Sinyal "teeet" dengan Hamming Window



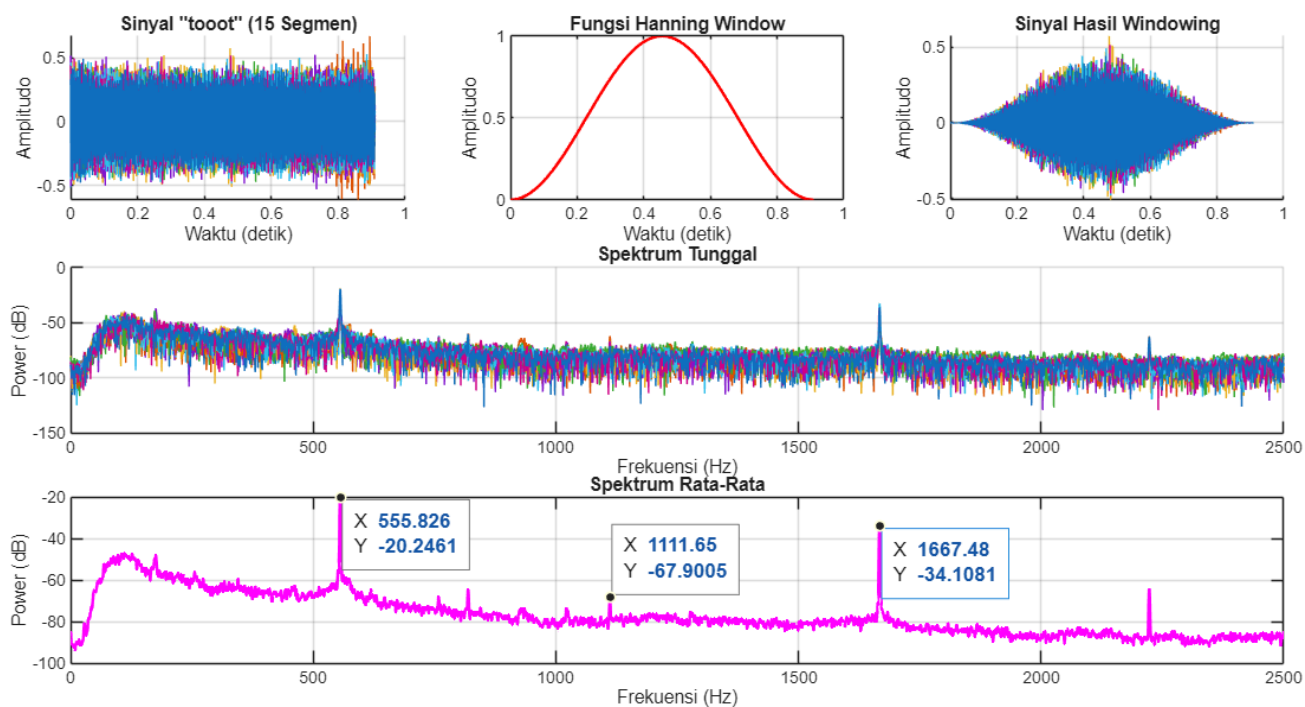
Gambar 6 - Analisis PSD Sinyal "teeet" dengan Hanning Window



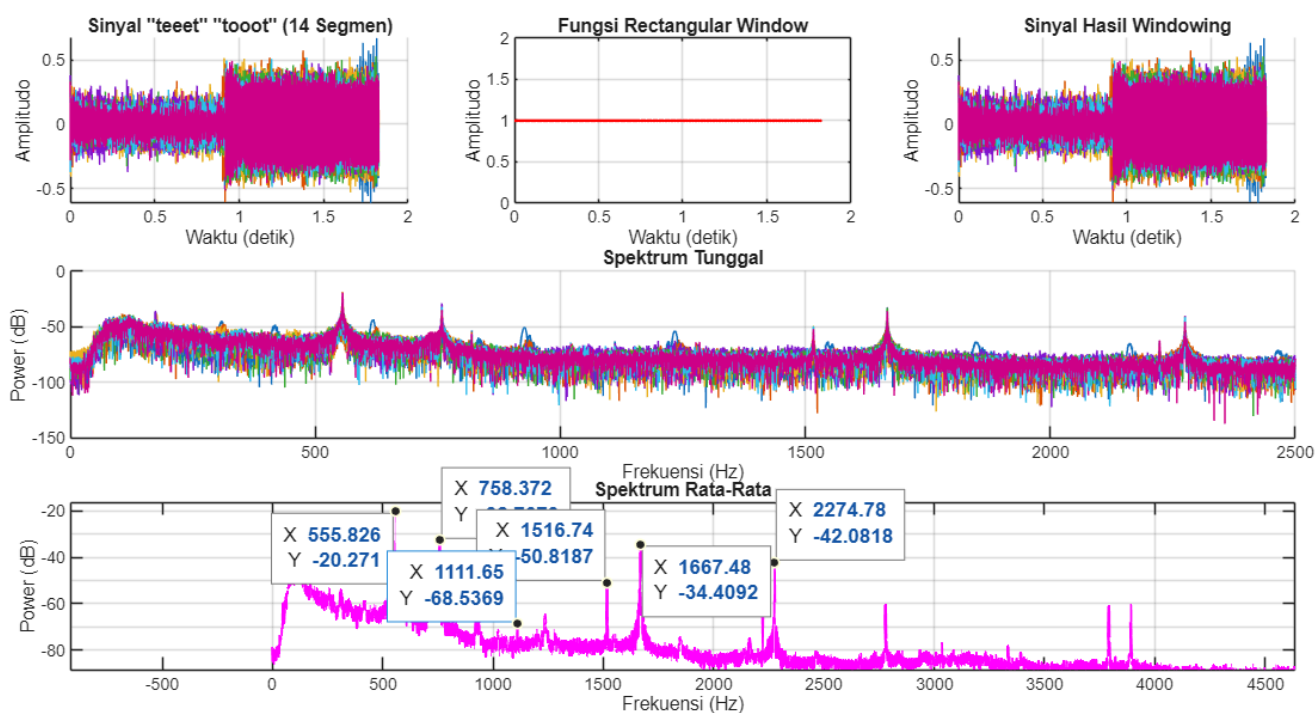
Gambar 7 - Analisis PSD Sinyal "toot" dengan Rectangular Window



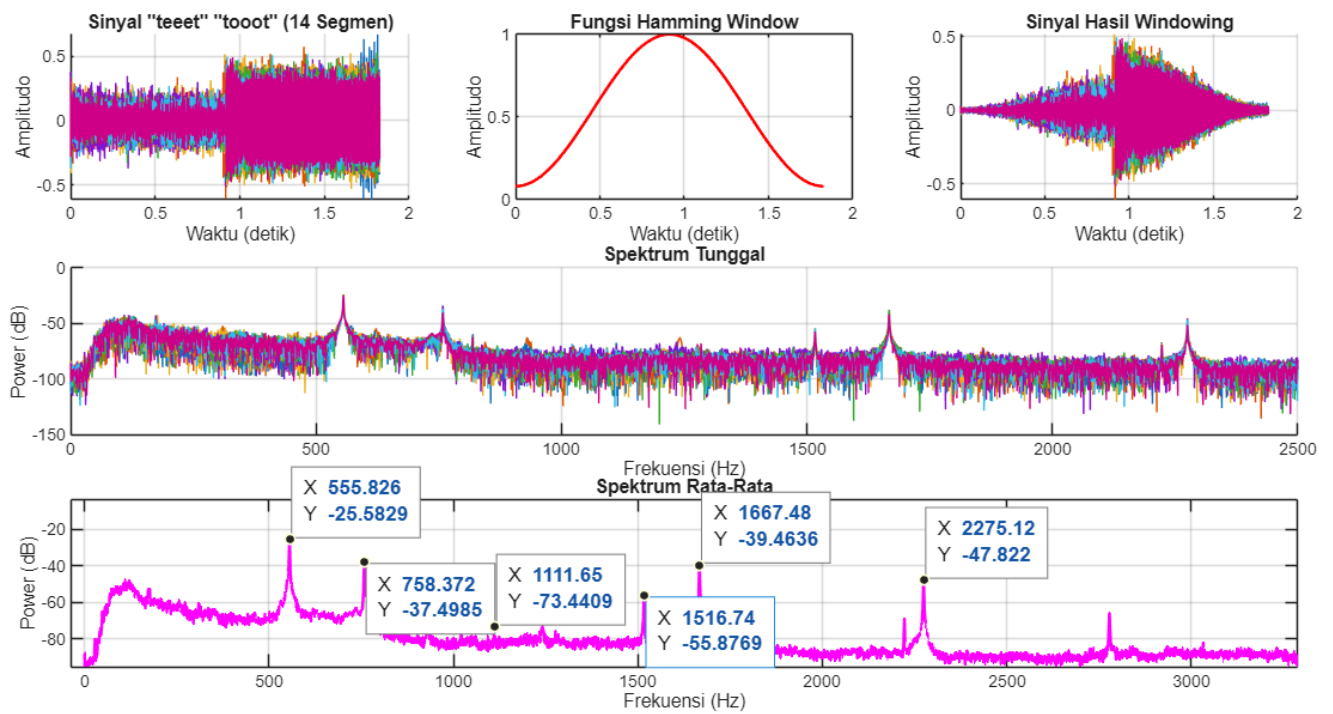
Gambar 8 - Analisis PSD Sinyal "toot" dengan Hamming Window



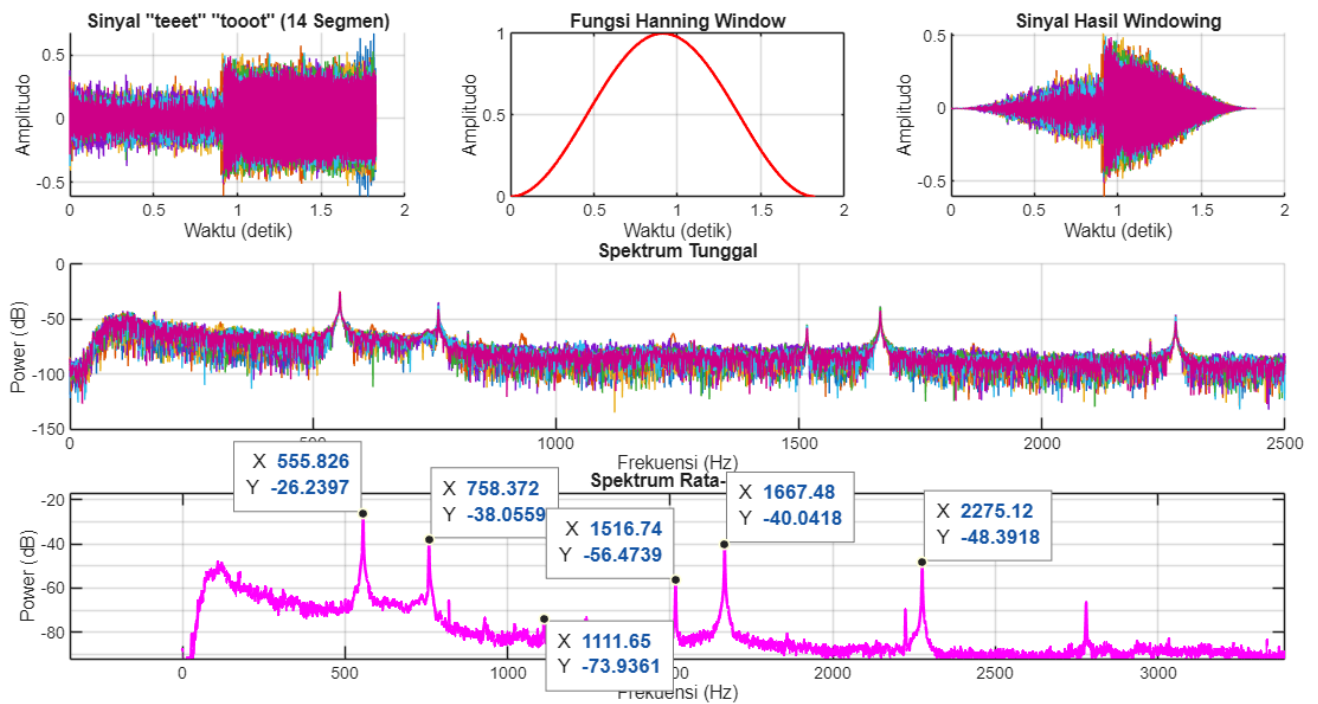
Gambar 9 - Analisis PSD Sinyal "toot" dengan Hanning Window



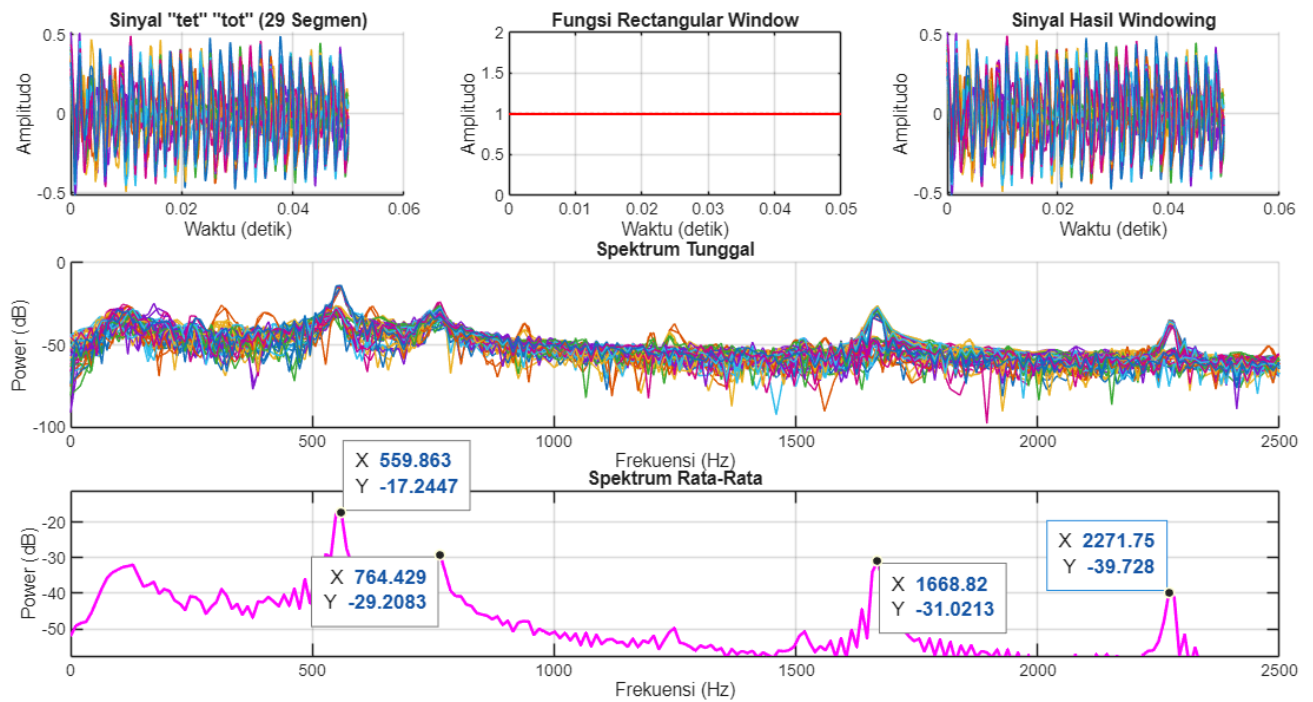
Gambar 10 - Analisis PSD Sinyal "teet" "toot" dengan Rectangular Window



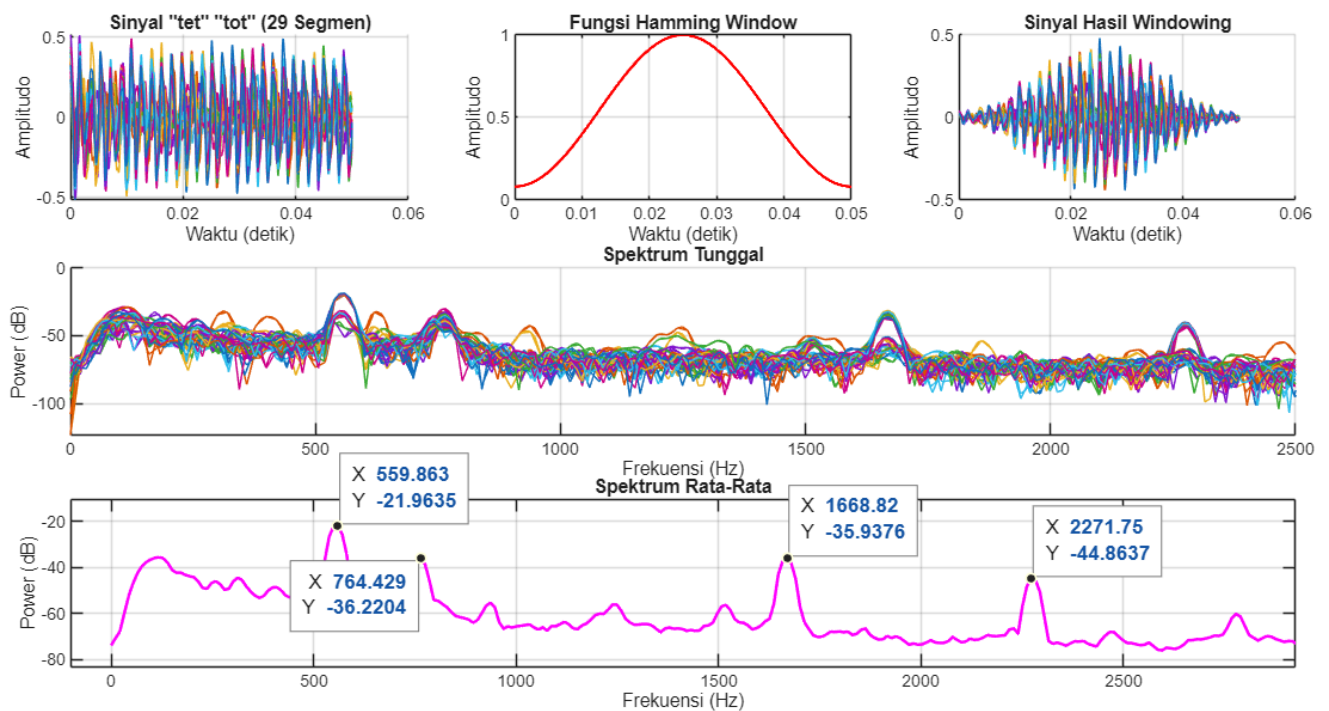
Gambar 11 - Analisis PSD Sinyal "teeet" "tooot" dengan Hamming Window



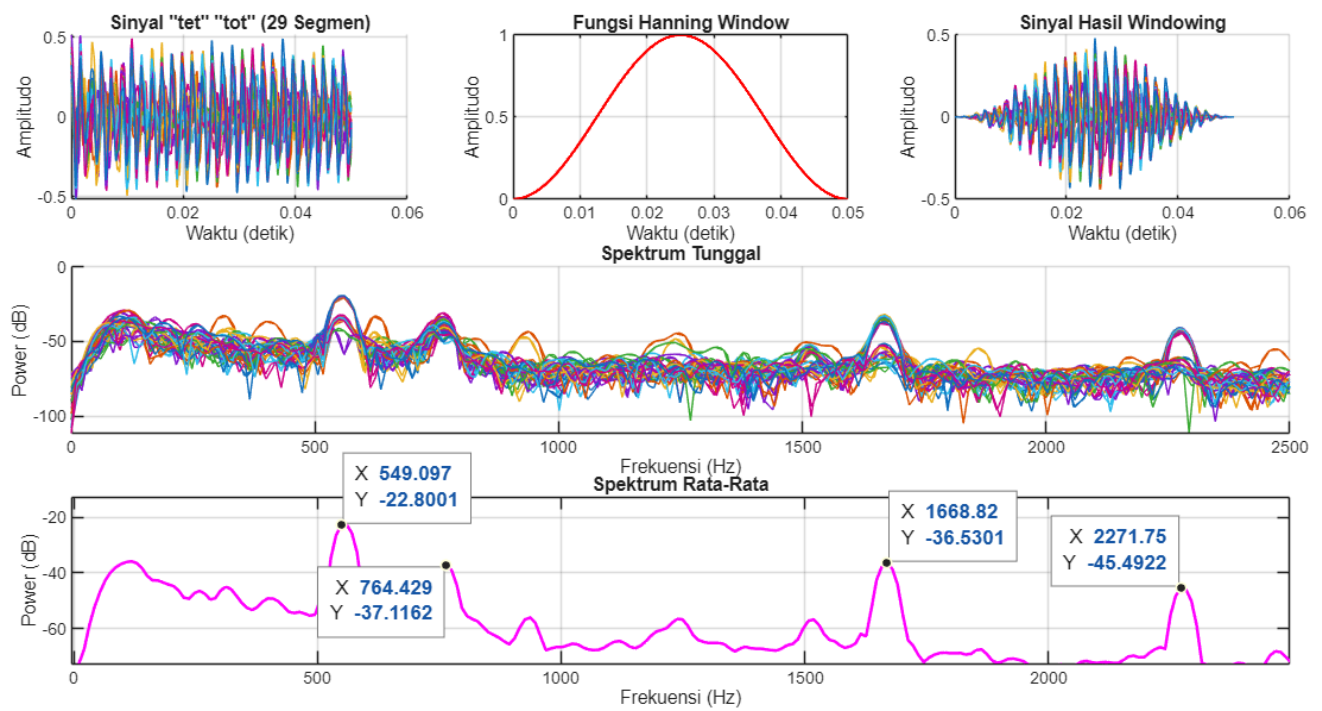
Gambar 12 - Analisis PSD Sinyal "teeet" "tooot" dengan Hanning Window



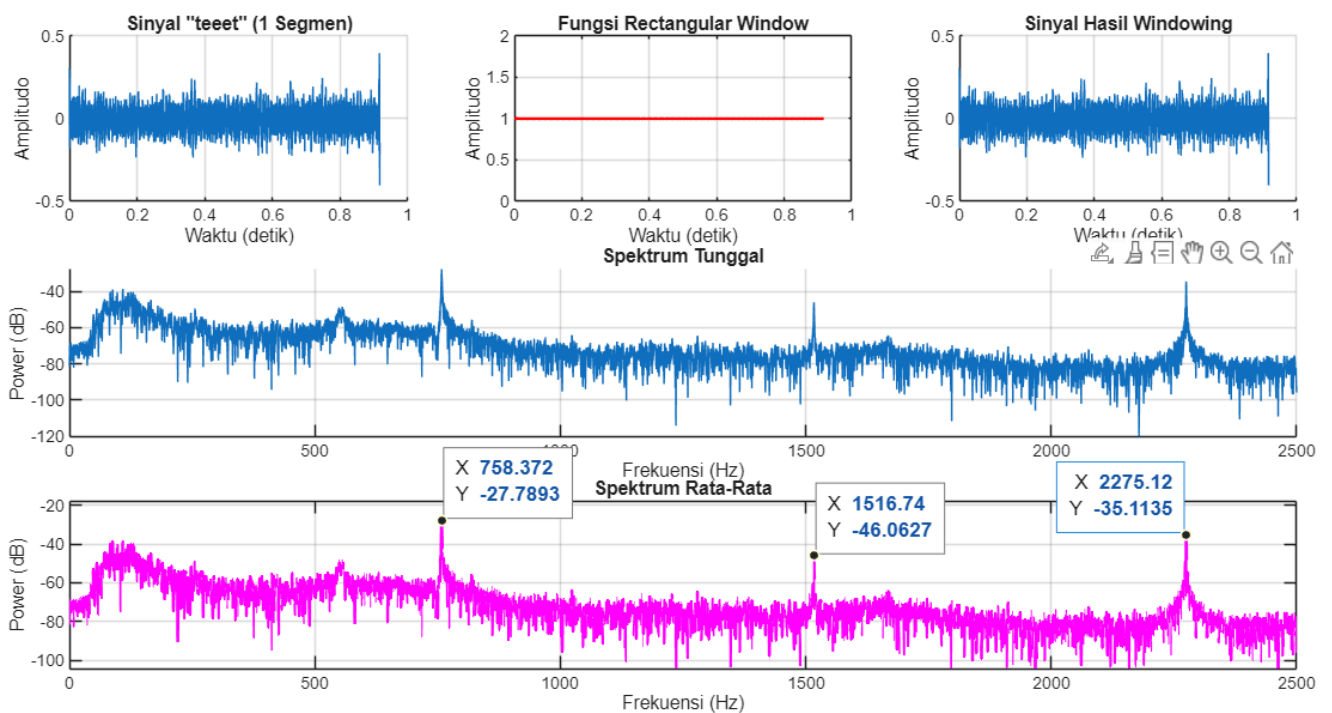
Gambar 13 - Analisis PSD Sinyal "tet" "tot" dengan Rectangular Window



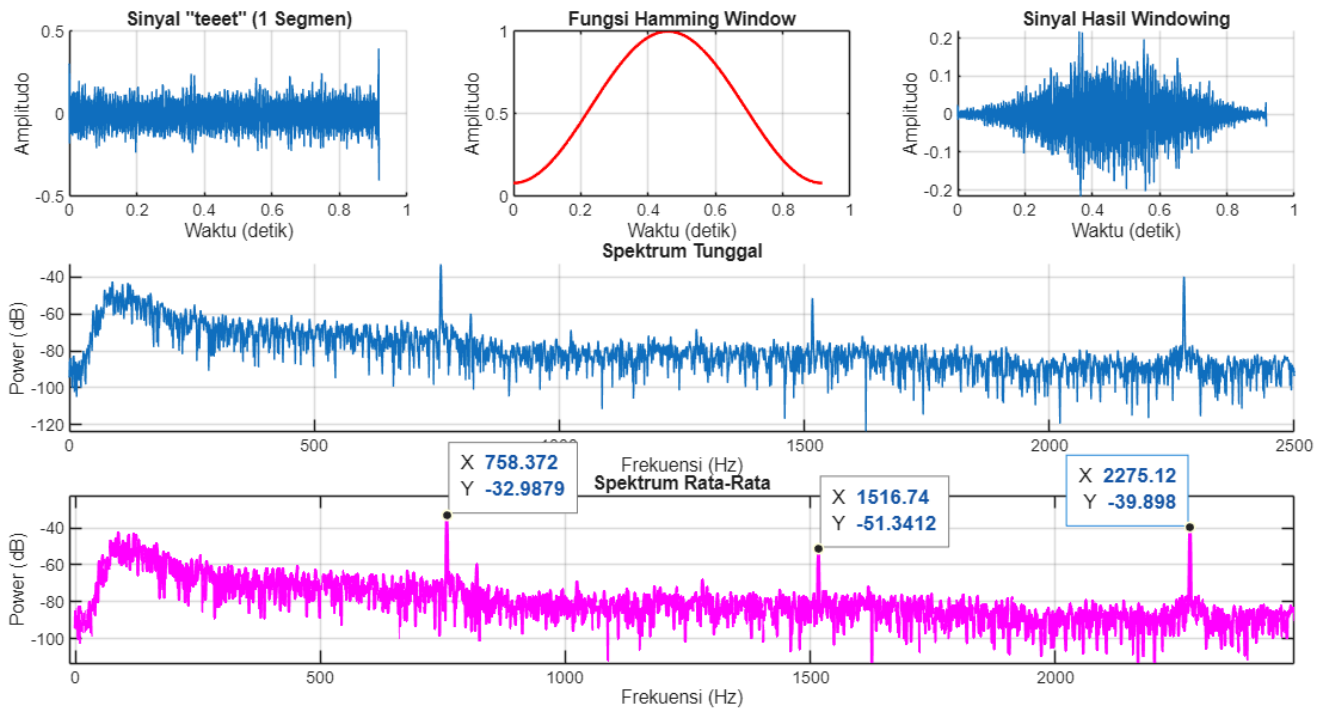
Gambar 14 - Analisis PSD Sinyal "tet" "tot" dengan Hamming Window



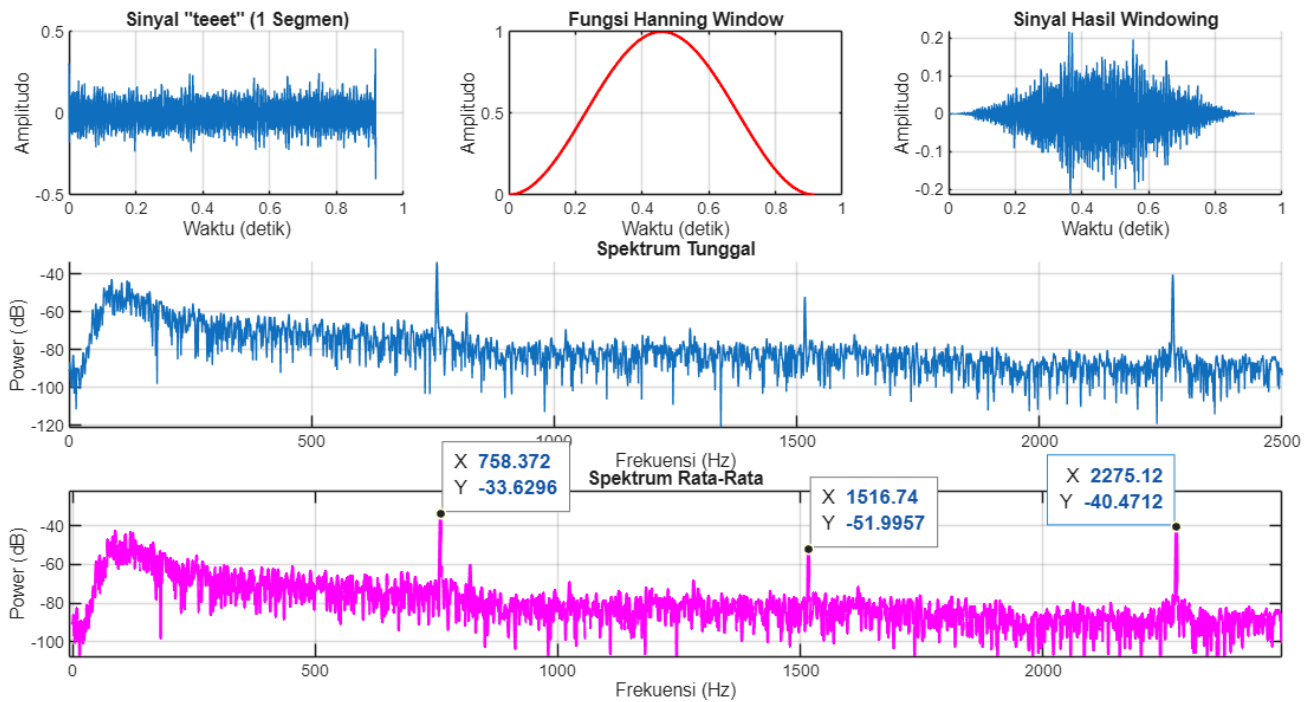
Gambar 15 - Analisis PSD Sinyal "tet" "tot" dengan Hanning Window



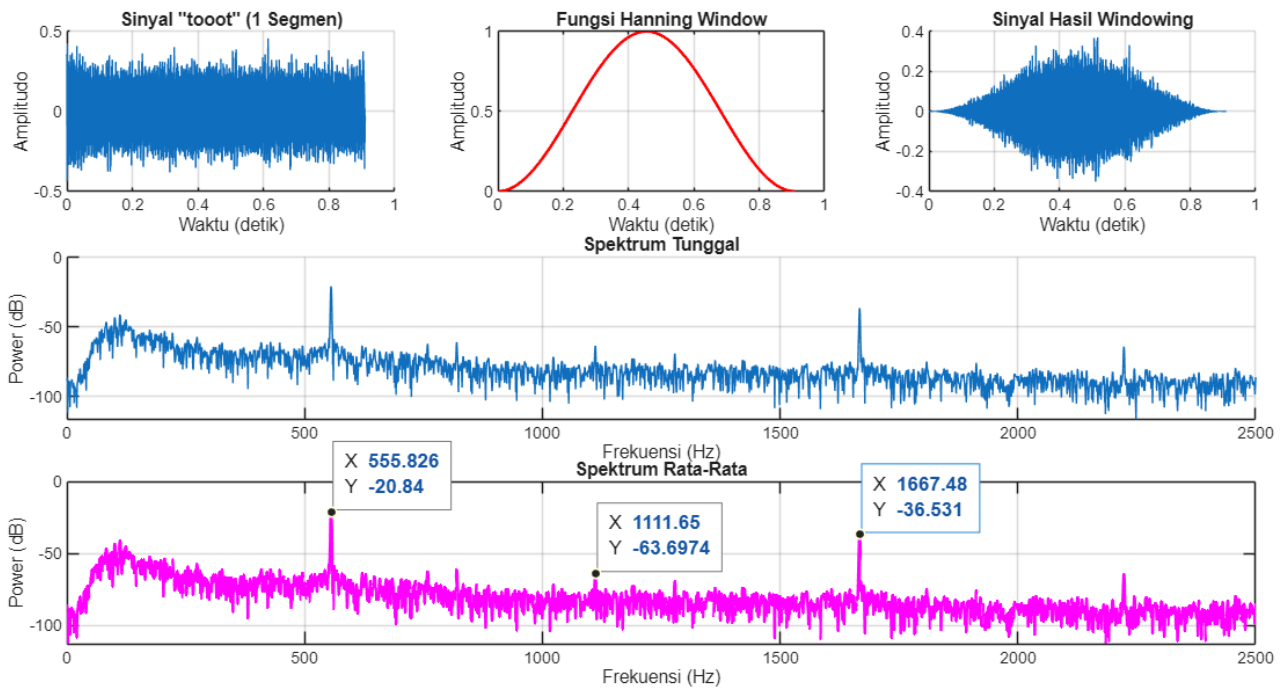
Gambar 16 - Analisis PSD Sinyal "teeet" Satu Segmen dengan Rectangular Window



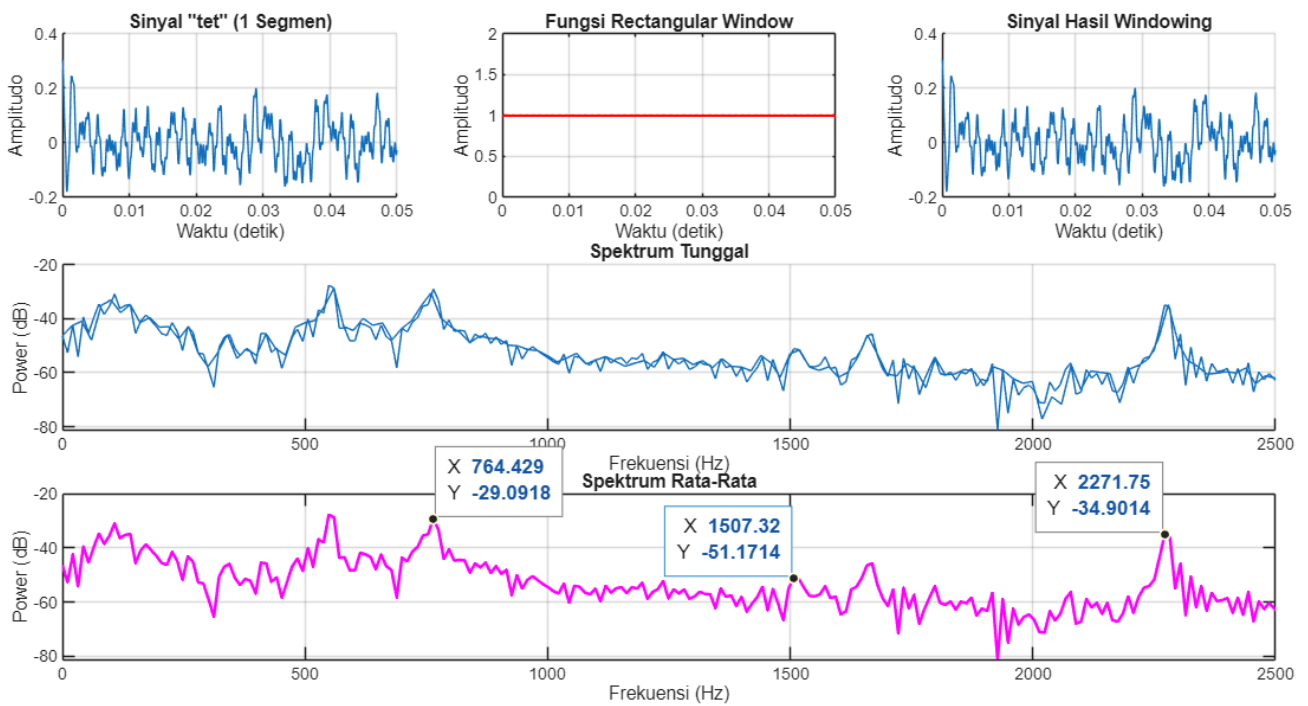
Gambar 17 - Analisis PSD Sinyal "teeet" Satu Segmen dengan Hamming Window



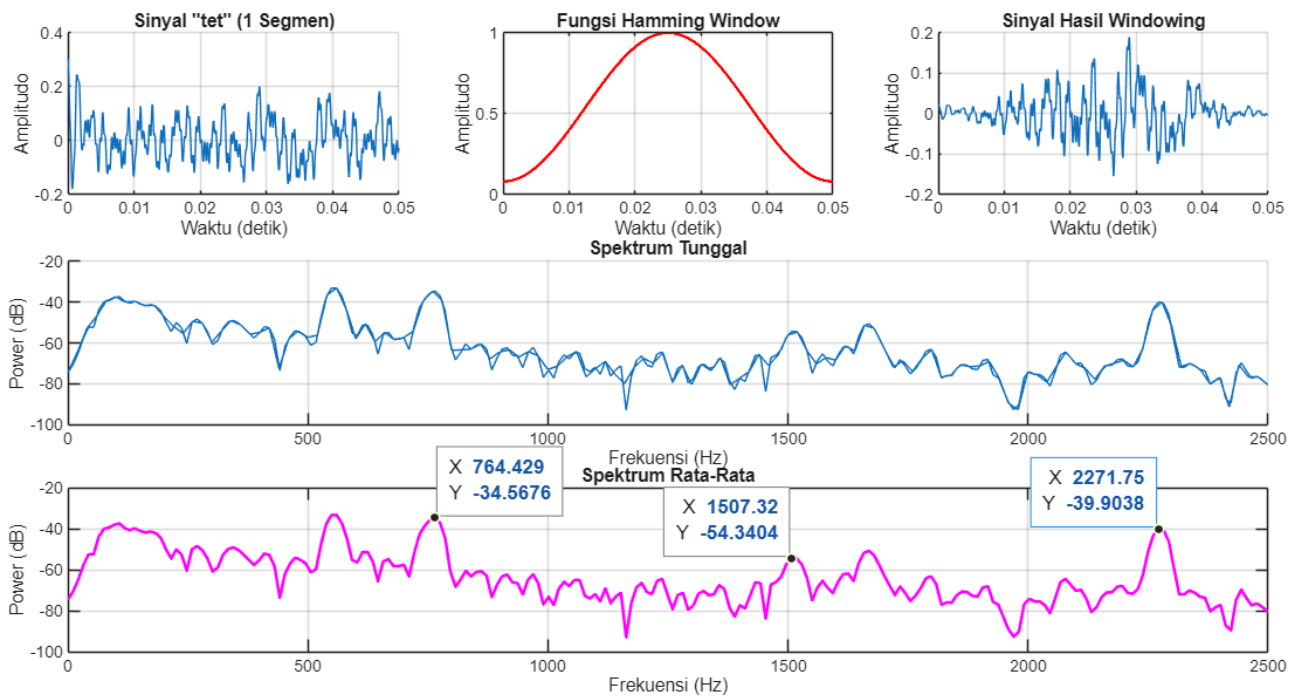
Gambar 18 - Analisis PSD Sinyal "teeet" Satu Segmen dengan Hanning Window



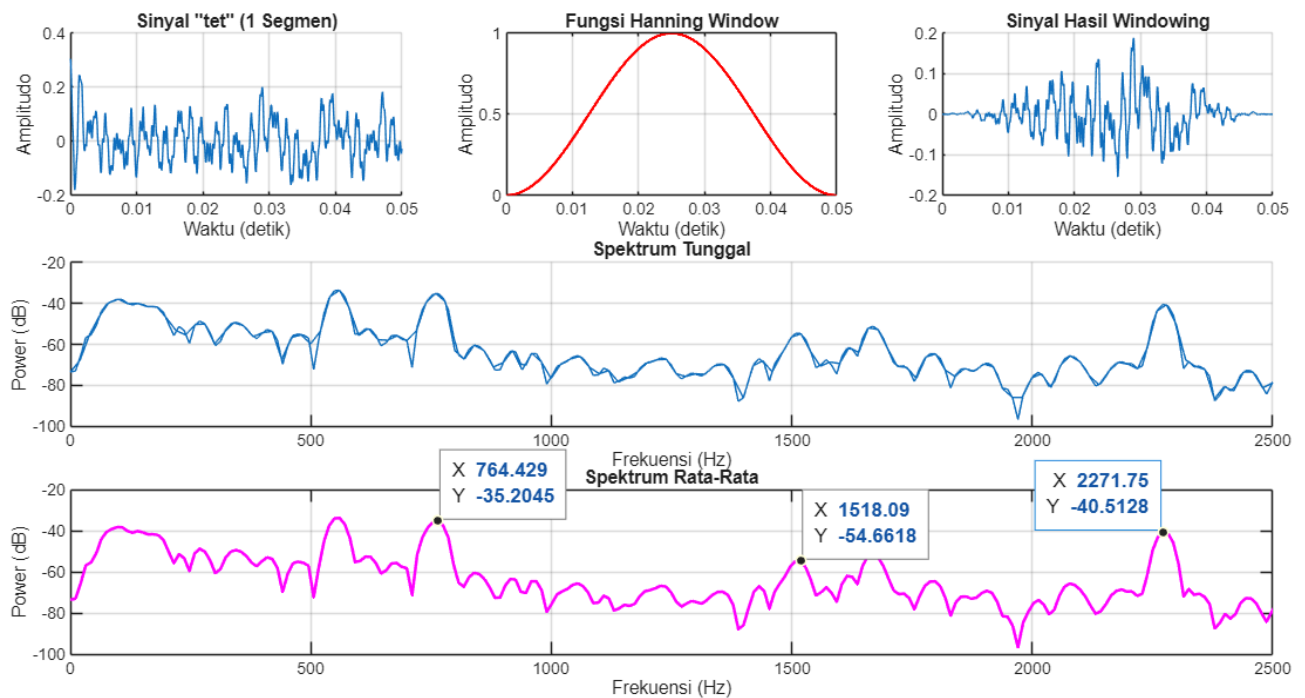
Gambar 21 - Analisis PSD Sinyal "toot" Satu Segmen dengan Hanning Window



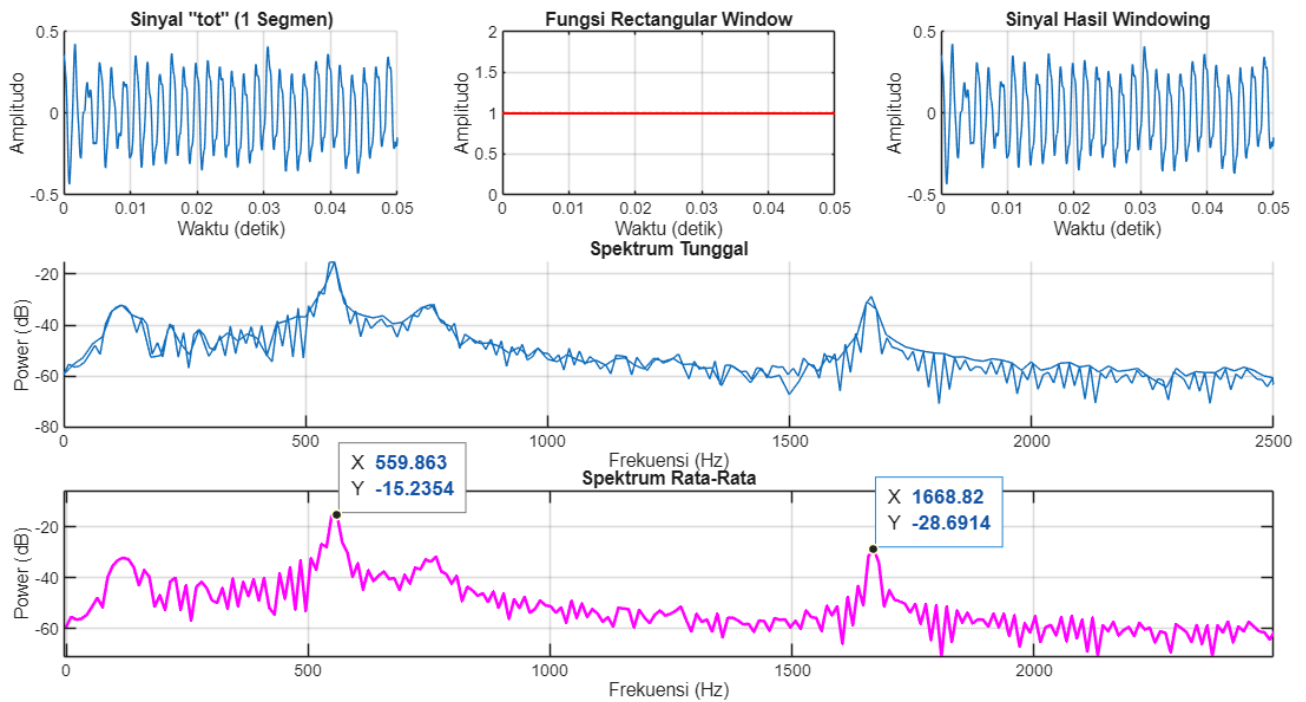
Gambar 22 - Analisis PSD Sinyal "tet" Satu Segmen dengan Rectangular Window



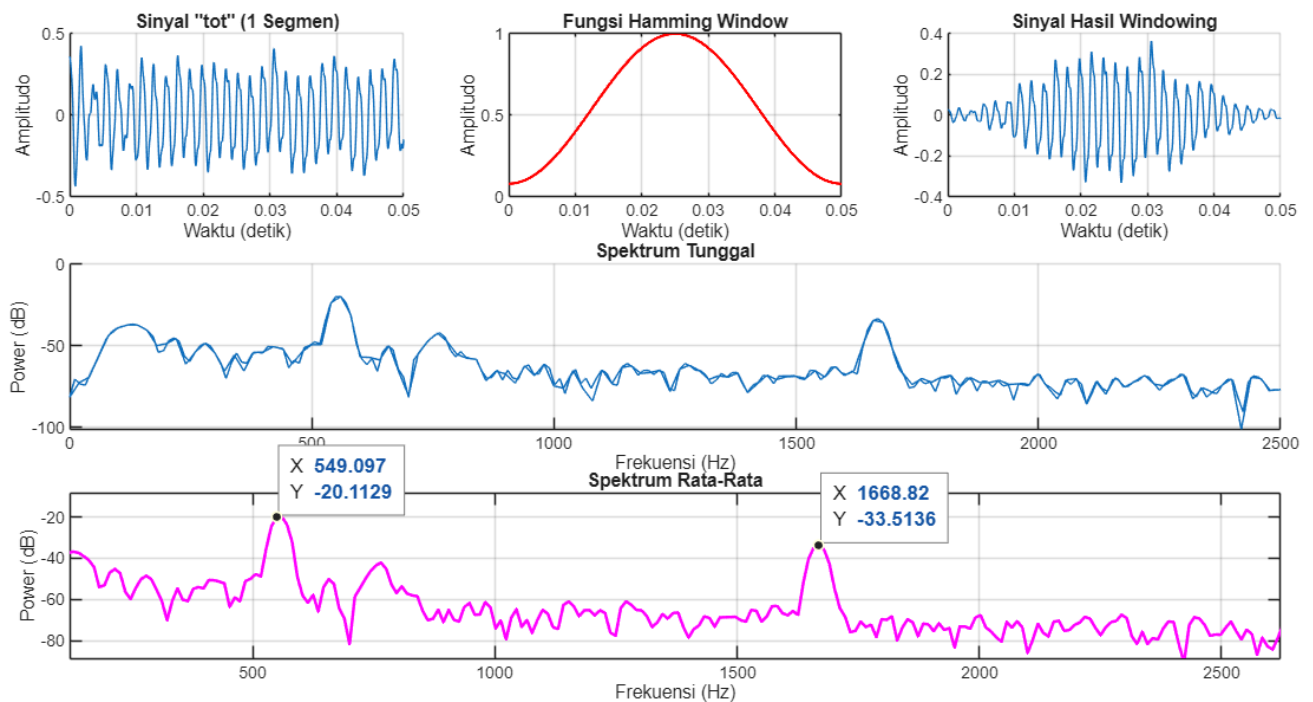
Gambar 23 - Analisis PSD Sinyal "tet" Satu Segmen dengan Hamming Window



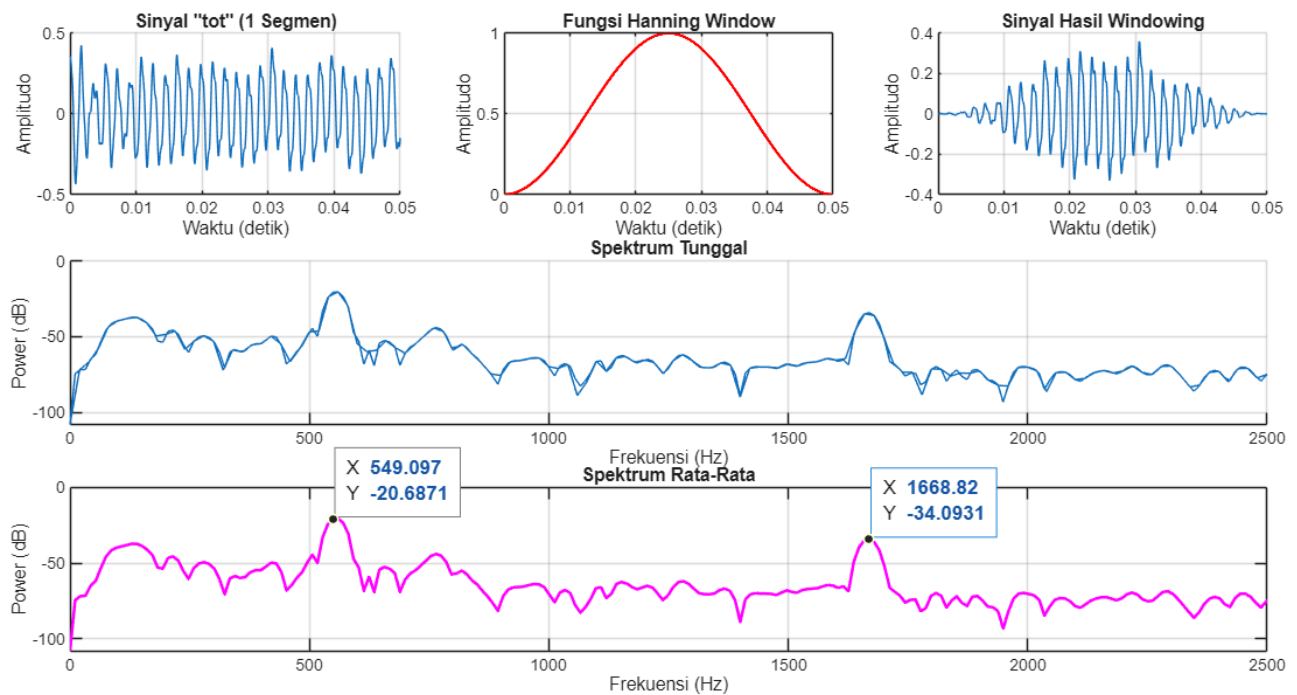
Gambar 24 - Analisis PSD Sinyal "tet" Satu Segmen dengan Hanning Window



Gambar 25 - Analisis PSD Sinyal "tot" Satu Segmen dengan Rectangular Window



Gambar 26 - Analisis PSD Sinyal "tot" Satu Segmen dengan Hamming Window



Gambar 27 - Analisis PSD Sinyal "tot" Satu Segmen dengan Hanning Window

```

Command Window
Analisis PSD Sinyal teeet

FFT dengan Rectangular Window
Waktu FFT tanpa zero padding = 0.107 detik
Waktu FFT dengan zero padding = 0.18759 detik

FFT dengan Hamming Window
Waktu FFT tanpa zero padding = 0.073161 detik
Waktu FFT dengan zero padding = 0.09409 detik

FFT dengan Hanning Window
Waktu FFT tanpa zero padding = 0.078882 detik
Waktu FFT dengan zero padding = 0.076994 detik

Analisis PSD Sinyal toot

FFT dengan Rectangular Window
Waktu FFT tanpa zero padding = 0.089207 detik
Waktu FFT dengan zero padding = 0.067421 detik

FFT dengan Hamming Window
Waktu FFT tanpa zero padding = 0.090091 detik
Waktu FFT dengan zero padding = 0.092691 detik

FFT dengan Hanning Window
Waktu FFT tanpa zero padding = 0.090545 detik
Waktu FFT dengan zero padding = 0.080974 detik
  
```

Gambar 28 - Contoh Hasil Pengukuran Waktu Pemrosesan FFT dengan Zero Padding dan Tanpa Zero Padding

Gambar di atas merupakan hasil PSD pada sinyal:

- “teeet” (Gambar 4 s.d. 6 untuk seluruh segmen, Gambar 16 s.d. 18 untuk satu segmen)
- “tooot” (Gambar 7 s.d. 9 untuk seluruh segmen, Gambar 19 s.d. 21 untuk satu segmen)
- “teeet.. tooot..” (Gambar 10 s.d. 12)
- “tet.. tot..” (Gambar 13 s.d. 15)
- “tet” (Gambar 22 s.d. 24)
- “tot” (Gambar 25 s.d. 27)

Masing-masing suara dianalisis dengan *rectangular window*, *hamming window* dan *hanning window*.

Tahap pertama untuk menampilkan PSD adalah pemotongan sinyal menjadi sekumpulan sinyal, misalnya “teeet”. Berdasarkan soal nomor 1a, sinyal “teeet” terdapat pada durasi $t = 1.60646, 3.43342, 5.26565, 7.08896, 8.91415, 10.7427, 11.6568, 13.4838, 15.3159, 18.0407, 19.8694, 21.6945, 23.525, 25.3521$ dengan durasi 0.9169. Setiap segmen sinyal dipotong sesuai durasi tersebut, dikumpulkan kemudian dikalikan dengan *window*. Dengan menggunakan *window*, amplitudo di awal dan akhir segmen diredam menuju nol, sehingga sinyal tetap terfokus pada frekuensi dominannya dan tidak melebar ke frekuensi sekitarnya. Selanjutnya, *windowed signal* pada domain waktu ditransformasikan ke domain frekuensi menggunakan *Fast Fourier Transform* (FFT). Hasil tersebut kemudian dikonversi menjadi nilai densitas daya pada setiap komponen frekuensi untuk membentuk grafik *Power Spectral Density* (PSD). Sesuai dengan *Central Limit Theorem* (CLT), spektrum dari beberapa segmen sinyal kemudian dirata-ratakan. Proses ini bertujuan untuk menekan variansi pada estimasi spektrum, sehingga menghasilkan grafik spektrum rata-rata yang lebih stabil dan representatif terhadap karakteristik sinyal sebenarnya.

Berdasarkan Gambar 5 dan 8, sinyal “teeet” dan “tooot” menunjukkan perbedaan karakteristik frekuensi meskipun keduanya memiliki resolusi frekuensi yang tinggi karena durasi sinyal yang panjang yaitu 0,9 detik. Sinyal “teeet” memiliki frekuensi fundamental sebesar 758,37 Hz dengan puncak harmonik yang terdeteksi secara presisi pada 1516,74 Hz dan 2275,12 Hz. Di sisi lain, sinyal “tooot” memiliki nada dasar yang lebih rendah dengan frekuensi fundamental sebesar 555,82 Hz dan komponen harmonik pada 1111,65 Hz serta 1667,48 Hz. Secara visual, puncak daya pada sinyal “tooot” (-19,58 dB) terlihat lebih dominan dibandingkan sinyal “teeet” (-32,23 dB).

Tabel 1 - Analisis Frekuensi Sinyal “teeet” dan “tooot” Dengan Hamming Window (Gambar 5 dan Gambar 8)

Parameter Analisis	Sinyal “teeet” (14 Segmen)	Sinyal “tooot” (15 Segmen)
Frekuensi Fundamental	758.37 Hz (-32.23 dB)	555.82 Hz (-19.58 dB)
Frekuensi Harmonik ke-2	1516.74 Hz (-49.85 dB)	1111.65 Hz (-67.43 dB)
Frekuensi Harmonik ke-3	2275.12 Hz (-41.63 dB)	1667.48 Hz (-33.50 dB)

Untuk melihat efek dari CLT, kita dapat membandingkan spektrum 1 segmen dengan 14 segmen. Spektrum 1 segmen menampilkan grafik yang sangat kasar dan *noisy* karena hanya mengandalkan satu observasi acak yang rentan terhadap fluktuasi sesaat. Hal ini menyebabkan nilai daya (dB) pada setiap puncak frekuensi cenderung tidak stabil dan memiliki penyimpangan yang lebih besar. Sebaliknya, pada 14 segmen, spektral rata-rata membuktikan CLT yaitu dengan merata-ratakan banyak sampel independen, fluktuasi acak tersebut saling meniadakan sehingga *noise* menjadi lebih halus dan estimasi daya pada setiap komponen frekuensi menjadi lebih stabil dan presisi.

Tabel 2 - Analisis Frekuensi Sinyal “teeet” dan “tooot” Dengan Hamming Window (Gambar 17 dan Gambar 20)

Parameter Analisis	Sinyal “teeet” (1 Segmen)	Sinyal “tooot” (1 Segmen)
--------------------	---------------------------	---------------------------

Frekuensi Fundamental	758.37 Hz (-32.98 dB)	555.82 Hz (-20.19 dB)
Frekuensi Harmonik ke-2	1516.74 Hz (-51.34 dB)	1111.65 Hz (-63.20 dB)
Frekuensi Harmonik ke-3	2275.12 Hz (-39.89 dB)	1667.48 Hz (-35.89 dB)

Dalam melakukan *Discrete Fourier Transform* (DFT), zero padding berfungsi untuk memperhalus tampilan spektrum di domain frekuensi melalui interpolasi nilai antar *bin*. Berdasarkan Gambar 28, dengan menggunakan *zero padding* komputer harus memproses jumlah data yang lebih besar untuk mencapai panjang sinyal 2^n terdekat yaitu 2^{16} atau 65.536 sampel, sehingga waktu komputasi menjadi lebih lama. Hal ini terlihat pada pengujian sinyal "teet" dengan *hamming window*, di mana waktu proses meningkat dari 0,073 detik menjadi 0,094 detik. Sebaliknya, dengan tidak menggunakan *zero padding*, proses FFT berjalan lebih cepat karena algoritma hanya mengolah jumlah sampel asli sebesar 40.440 sampel (hasil durasi ~0,917 detik dikalikan F_s 44.100 Hz), sehingga meminimalkan beban komputasi.

Dari hasil perbandingan 3 window yaitu *rectangular*, *hamming* dan *hanning* didapatkan analisis sebagai berikut. *Rectangular window* menghasilkan PSD dengan *main lobe* yang paling sempit sehingga memberikan resolusi frekuensi terbaik atau mampu membedakan dua frekuensi yang sangat dekat. Namun, *side lobes* relatif tinggi sehingga menyebabkan kebocoran spektral yang besar. *Hanning window* menghasilkan PSD dengan *main lobe* yang kira-kira dua kali lebih lebar dibandingkan *rectangular window*, sehingga resolusinya berkurang. Namun, *side lobes*nya jauh lebih kecil sehingga mengurangi kebocoran frekuensi. Sedangkan *hamming window* menghasilkan PSD yang berbentuk kurva yang mirip dengan Hanning tetapi tidak benar-benar menyentuh angka nol pada ujung-ujungnya, sehingga menyeimbangkan antara resolusi dan kebocoran frekuensi. Maka, dapat disimpulkan bahwa:

- Rectangular window*: Resolusi menjadi tinggi, *trade-off* dengan kebocoran frekuensi tinggi
- Hamming window*: Seimbang antara resolusi dan kebocoran frekuensi
- Hanning window*: Resolusi menjadi lebih rendah, *trade-off* dengan kebocoran frekuensi rendah

Sinyal dengan durasi lama seperti "teet" atau "toot" menghasilkan estimasi spektrum yang lebih akurat dibandingkan sinyal berdurasi pendek seperti "tet" atau "tot". PSD menunjukkan *main lobe* pada sinyal berdurasi lama lebih tajam, sehingga frekuensi fundamental serta harmoniknya dapat terdeteksi lebih mudah. Hal ini berkaitan dengan resolusi frekuensi, di mana durasi yang lebih panjang secara teoretis memberikan resolusi yang lebih baik karena lebar *main lobe* pada *window* menjadi lebih sempit. Sebaliknya, sinyal berdurasi pendek sering kali menghasilkan grafik spektrum yang sangat kasar dan *noisy* karena hanya mengandalkan sedikit informasi yang rentan terhadap fluktuasi acak. Selain itu, durasi yang lebih lama memungkinkan penerapan *Central Limit Theorem* (CLT) dari banyak segmen, yang menekan variansi noise sehingga menghasilkan estimasi daya yang lebih stabil dan representatif terhadap karakteristik sinyal yang sebenarnya.

Diberikan sinyal "teet..toot..teet..toot..teet..toot.. ...", Anda diminta untuk:

- mendesain 2 filters yang masing-masing mengeluarkan sinyal "teet.. ..teet.. ..teet.. ..." dan sinyal "toot.. ..toot.. ..toot.. ...".

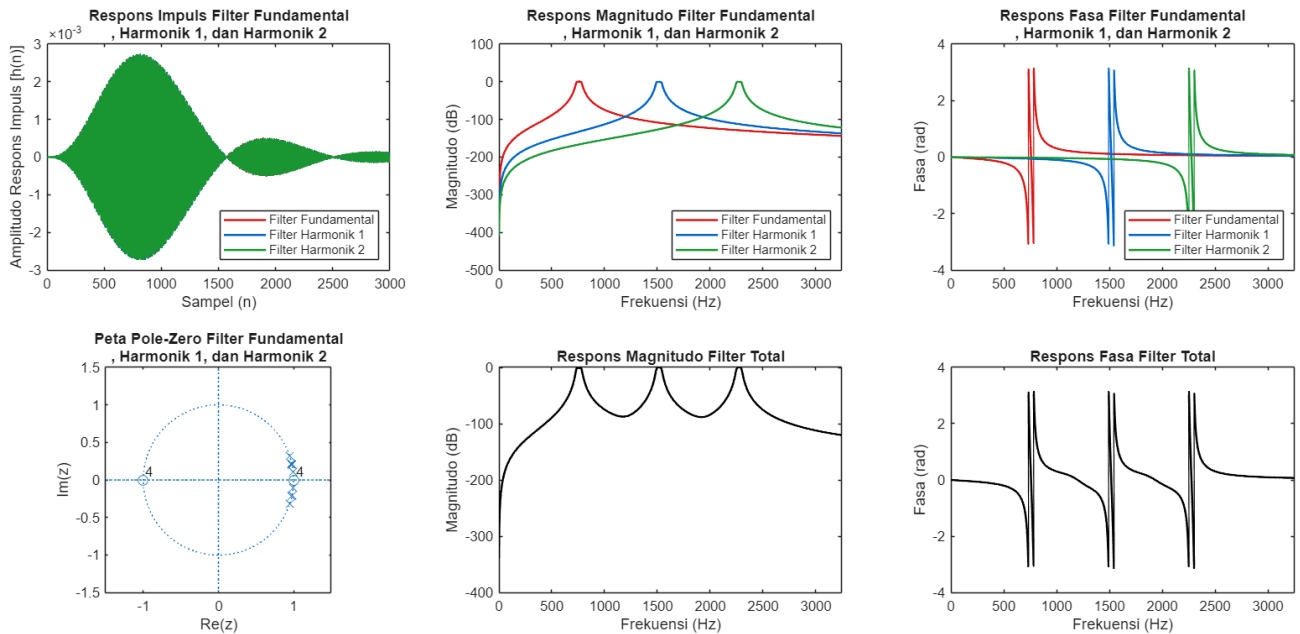
Tips: apa dasar Anda merancang spesifikasi filter tersebut?

bandingkan beberapa macam filter, seperti IIR Butterworth, IIR Chebyshev, dan Window-based FIR dengan menggunakan *built-in functions*, serta dengan penempatan manual pole-zero sederhana seperti didiskusikan di kelas.

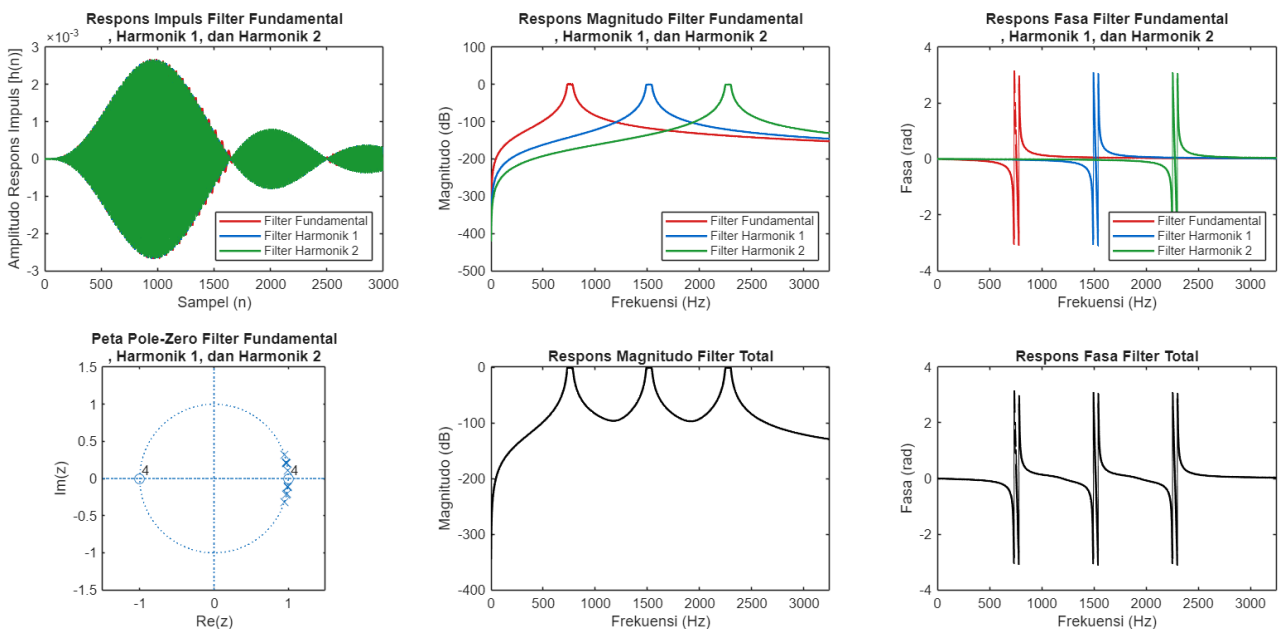
Tampilkan dan diskusikan filter tersebut dalam beberapa representasi sistem, seperti respons impuls, LCCDE, peta pole-zero, dan respons frekuensi.

Jawab:

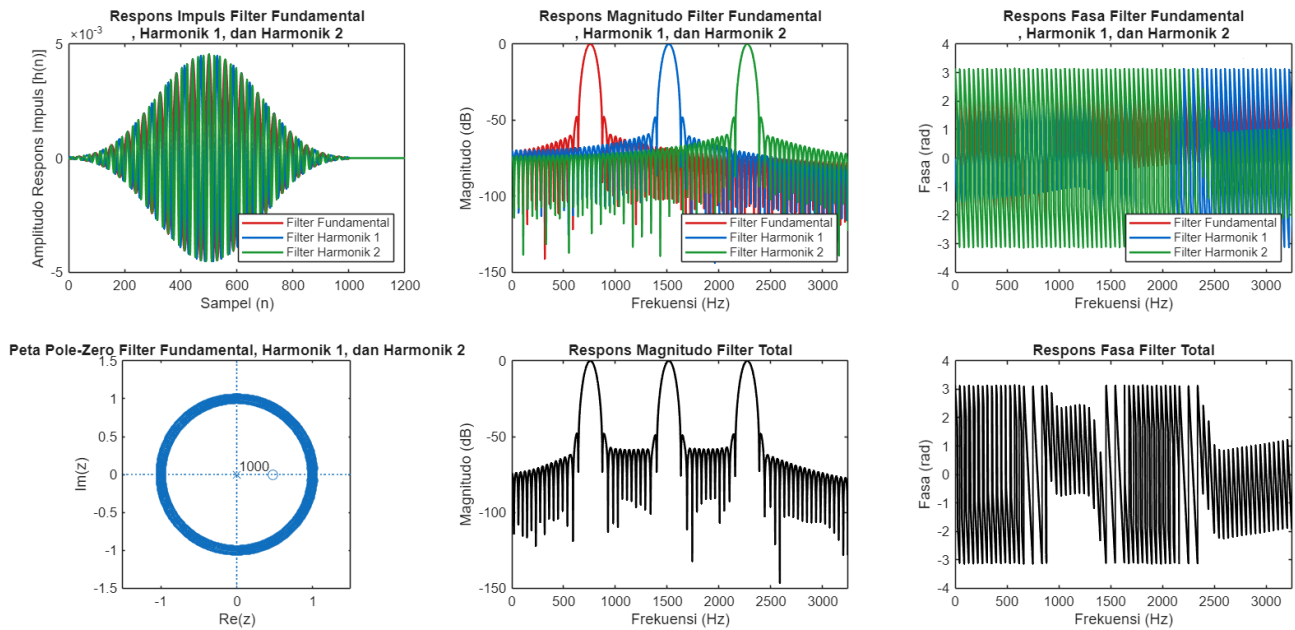
Pembuatan filter dapat dilaksanakan dengan bantuan kode Matlab [3] dan jika kode tersebut dijalankan, representasi sistem filter yang diperoleh adalah sebagai berikut.



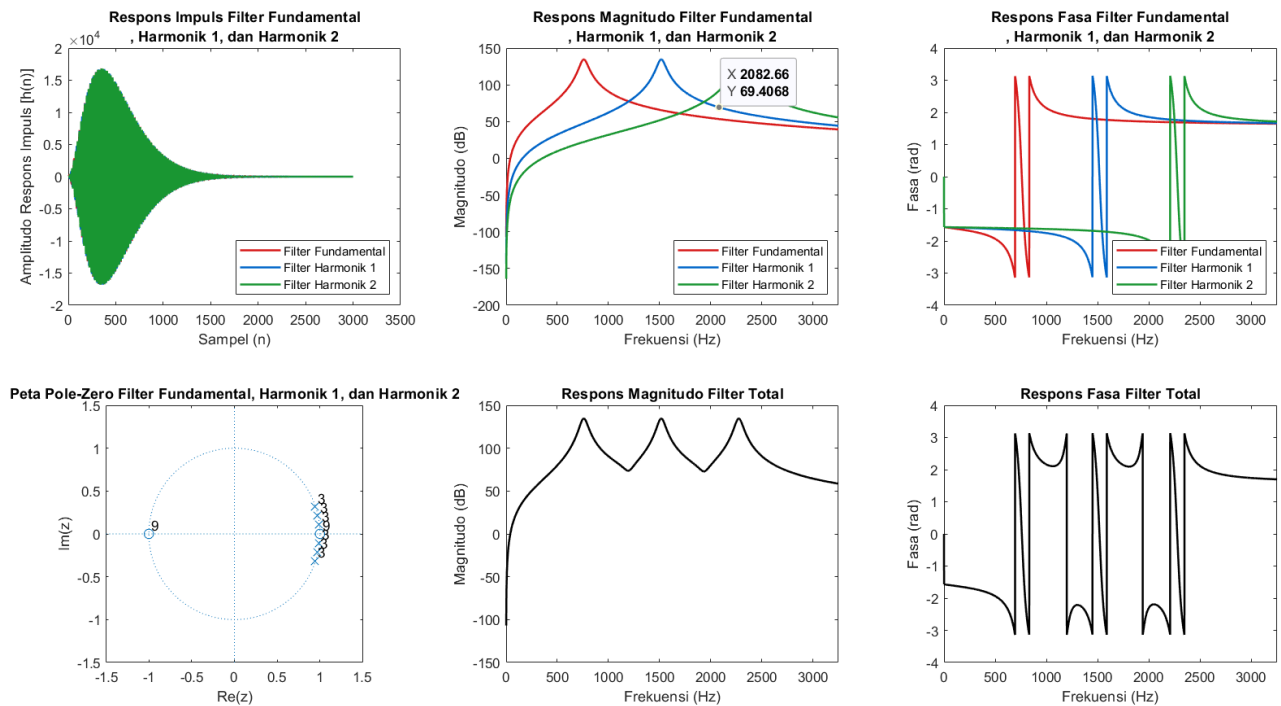
Gambar 29 - Representasi Sistem Filter Butterworth Built-In untuk Sinyal “teeet”



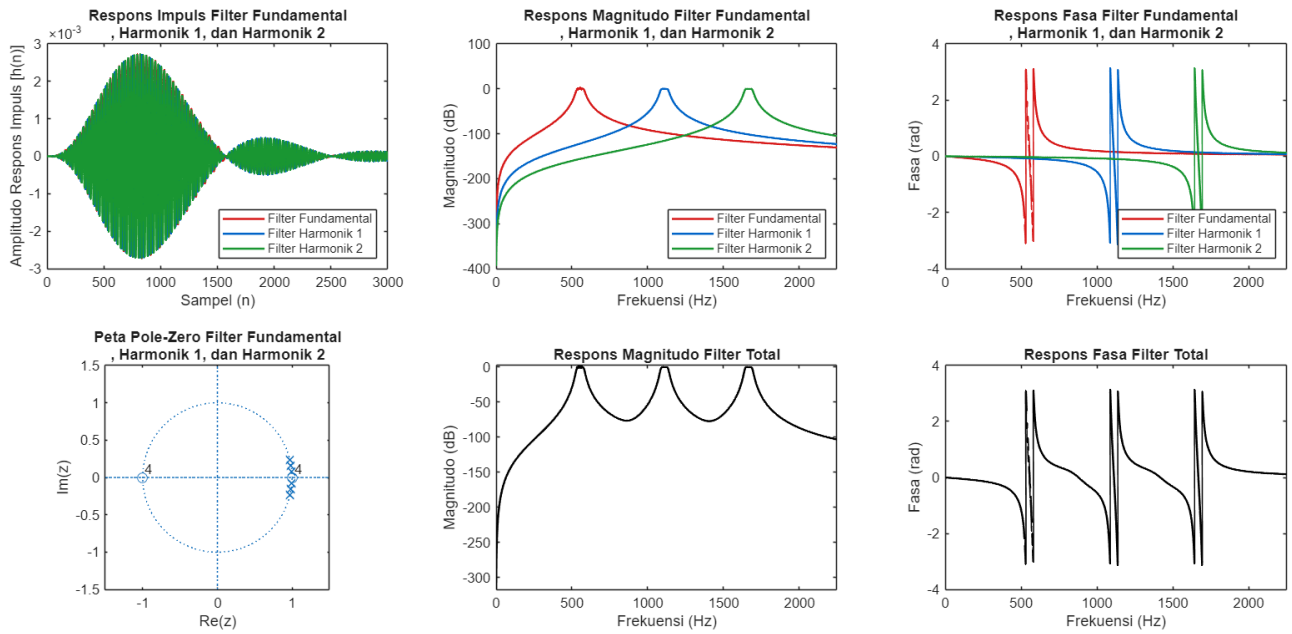
Gambar 30 - Representasi Sistem Filter Chebyshev1 Built-In untuk Sinyal “teeet”



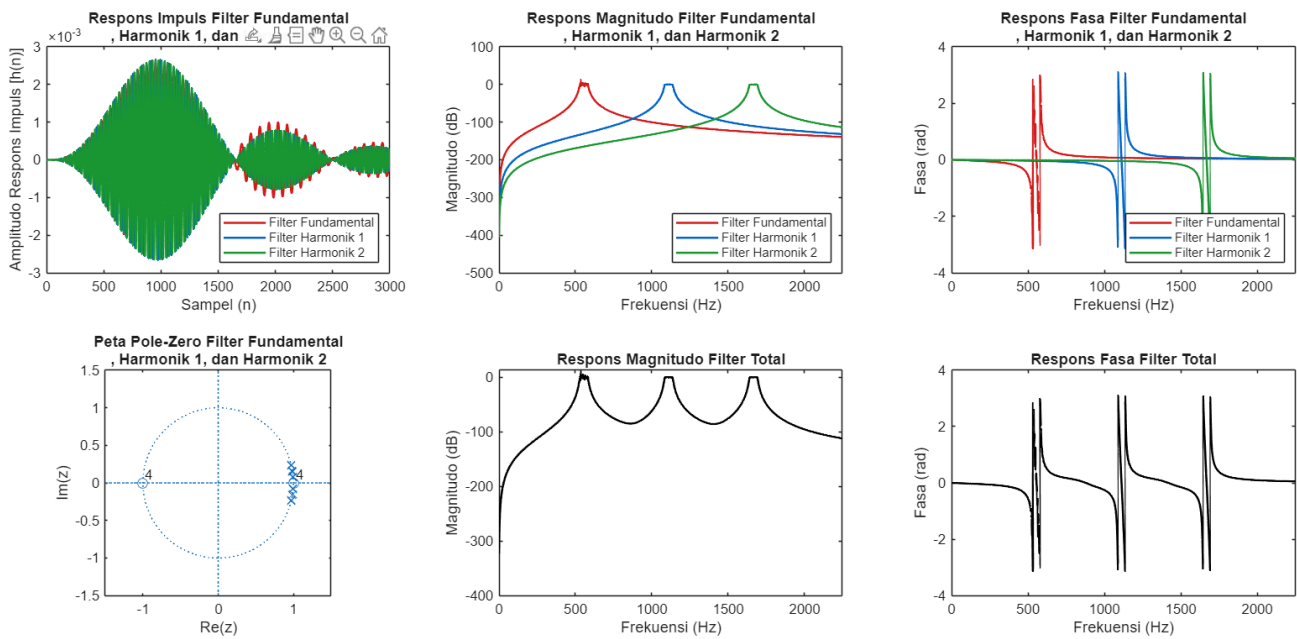
Gambar 31 - Representasi Sistem Filter FIR1 Built-In untuk Sinyal “teet”



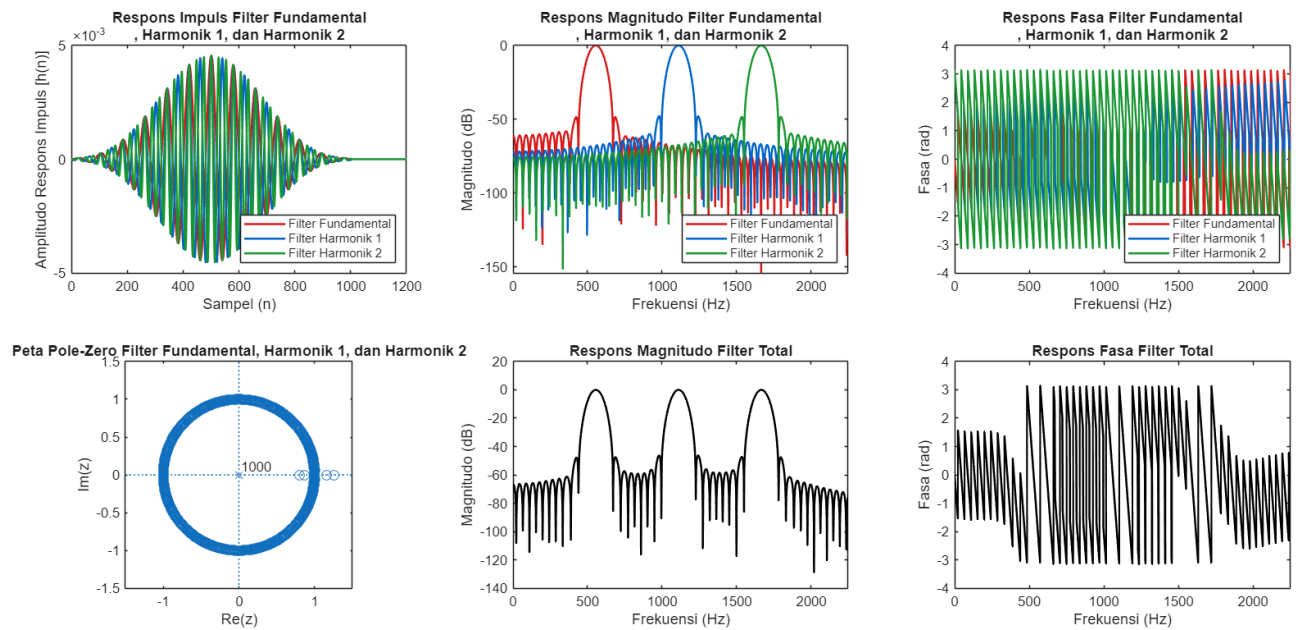
Gambar 32 - Representasi Sistem Filter dengan Peletakkan Manual Pole-Zero Sederhana untuk Sinyal “teet”



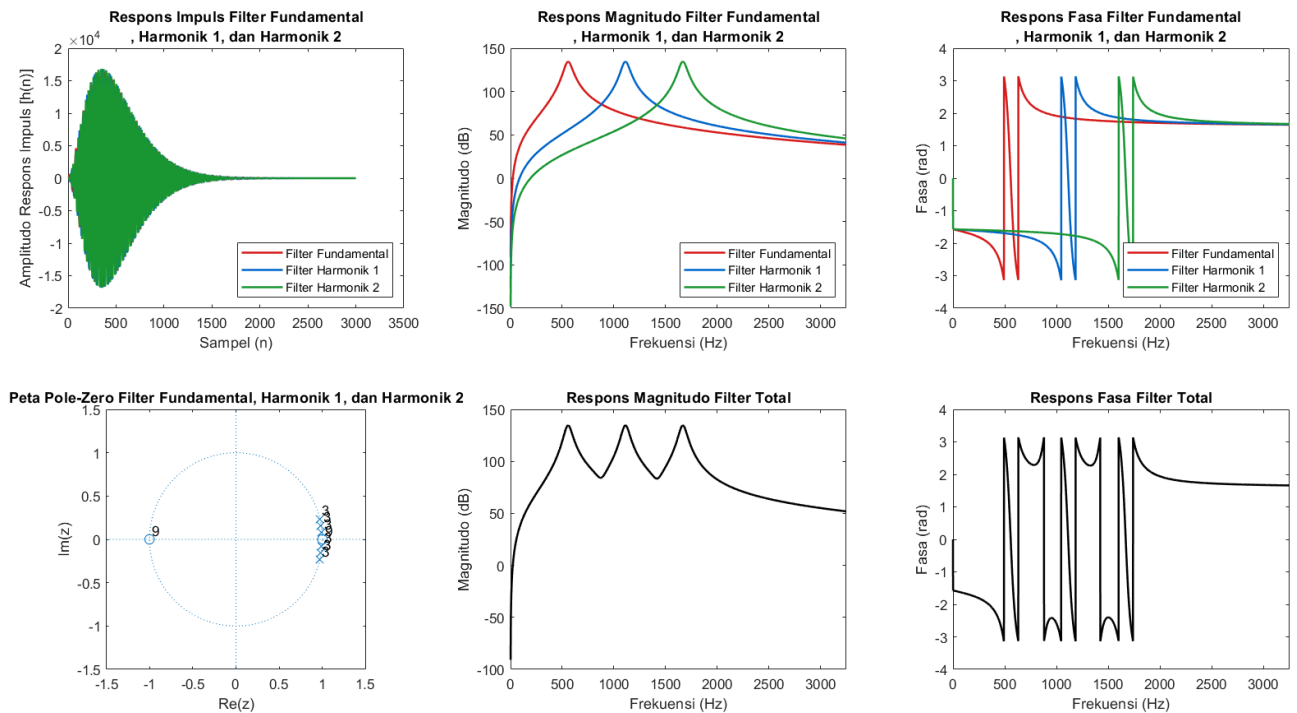
Gambar 33 - Representasi Sistem Filter Butterworth Built-In untuk Sinyal “toot”



Gambar 34 - Representasi Sistem Filter Chebyshev1 Built-In untuk Sinyal “toot”



Gambar 35 - Representasi Sistem Filter FIR1 Built-In untuk Sinyal “toot”



Gambar 36 - Representasi Sistem Filter dengan Peletakkan Manual Pole-Zero Sederhana untuk Sinyal “toot”

Desain Filter Sinyal teeet

Representasi Sistem Filter Butter Built-In
 Persamaan LCCDE Filter Frekuensi Fundamental
 $y(n) = (7.924)y[n-1] + (-27.52)y[n-2] + (54.69)y[n-3] + (-68.05)y[n-4] + (54.28)y[n-5] + (-27.11)y[n-6] + (7.749)y[n-7] + (-0.9707)y[n-8] + 1.039337$
 Persamaan LCCDE Filter Frekuensi Harmonik 1
 $y(n) = (7.785)y[n-1] + (-26.7)y[n-2] + (52.67)y[n-3] + (-65.38)y[n-4] + (52.28)y[n-5] + (-26.3)y[n-6] + (7.613)y[n-7] + (-0.9707)y[n-8] + 1.039337e-$
 Persamaan LCCDE Filter Frekuensi Harmonik 2
 $y(n) = (7.555)y[n-1] + (-25.38)y[n-2] + (49.45)y[n-3] + (-61.13)y[n-4] + (49.09)y[n-5] + (-25)y[n-6] + (7.388)y[n-7] + (-0.9707)y[n-8] + 1.039337e-0$

Representasi Sistem Filter Cheby1 Built-In
 Persamaan LCCDE Filter Frekuensi Fundamental
 $y(n) = (7.94)y[n-1] + (-27.63)y[n-2] + (55.02)y[n-3] + (-68.6)y[n-4] + (54.83)y[n-5] + (-27.44)y[n-6] + (7.859)y[n-7] + (-0.9864)y[n-8] + 3.749242e-$
 Persamaan LCCDE Filter Frekuensi Harmonik 1
 $y(n) = (7.801)y[n-1] + (-26.8)y[n-2] + (52.99)y[n-3] + (-65.9)y[n-4] + (52.81)y[n-5] + (-26.62)y[n-6] + (7.721)y[n-7] + (-0.9864)y[n-8] + 3.749242e-$
 Persamaan LCCDE Filter Frekuensi Harmonik 2
 $y(n) = (7.57)y[n-1] + (-25.48)y[n-2] + (49.75)y[n-3] + (-61.63)y[n-4] + (49.58)y[n-5] + (-25.3)y[n-6] + (7.493)y[n-7] + (-0.9864)y[n-8] + 3.749242e-$

Representasi Sistem Filter FIR1 Built-In
 Persamaan LCCDE Filter Frekuensi Fundamental
 $y(n) = - (2.996e-05)x[n] - (3.276e-05)x[n-1] - (3.527e-05)x[n-2] - (3.744e-05)x[n-3] - (3.924e-05)x[n-4] - (4.062e-05)x[n-5] - (4.156e-05)x[n-6] - ($
 Persamaan LCCDE Filter Frekuensi Harmonik 1
 $y(n) = (1.21e-05)x[n] + (1.967e-05)x[n-1] + (2.661e-05)x[n-2] + (3.257e-05)x[n-3] + (3.721e-05)x[n-4] + (4.026e-05)x[n-5] + (4.151e-05)x[n-6] + (4.0$
 Persamaan LCCDE Filter Frekuensi Harmonik 2
 $y(n) = (1.026e-05)x[n] - (1.551e-06)x[n-1] - (1.37e-05)x[n-2] - (2.489e-05)x[n-3] - (3.39e-05)x[n-4] - (3.965e-05)x[n-5] - (4.141e-05)x[n-6] - (3.88$

Representasi Sistem Filter Peletakkan Manual Pole-Zero Sederhana
 Persamaan LCCDE Filter Frekuensi Fundamental
 $y(n) = (5.931)y[n-1] - (14.69)y[n-2] + (19.45)y[n-3] - (14.52)y[n-4] + (5.797)y[n-5] - (0.9663)y[n-6] + (1)x[n] - (3)x[n-2] + (3)x[n-4] - (1)x[n-6]$
 Persamaan LCCDE Filter Frekuensi Harmonik 1
 $y(n) = (5.827)y[n-1] - (14.28)y[n-2] + (18.85)y[n-3] - (14.12)y[n-4] + (5.695)y[n-5] - (0.9663)y[n-6] + (1)x[n] - (3)x[n-2] + (3)x[n-4] - (1)x[n-6]$
 Persamaan LCCDE Filter Frekuensi Harmonik 2
 $y(n) = (5.655)y[n-1] - (13.63)y[n-2] + (17.88)y[n-3] - (13.47)y[n-4] + (5.527)y[n-5] - (0.9663)y[n-6] + (1)x[n] - (3)x[n-2] + (3)x[n-4] - (1)x[n-6]$

Gambar 37 - Contoh Representasi Sistem Filter Sinyal “teeet” dengan LCCDE

Desain Filter Sinyal tooot

Representasi Sistem Filter Butter Built-In
 Persamaan LCCDE Filter Frekuensi Fundamental
 $y(n) = (7.945)y[n-1] + (-27.64)y[n-2] + (55.01)y[n-3] + (-68.47)y[n-4] + (54.6)y[n-5] + (-27.24)y[n-6] + (7.77)y[n-7] + (-0.9707)y[n-8] + 1.039337e-$
 Persamaan LCCDE Filter Frekuensi Harmonik 1
 $y(n) = (7.871)y[n-1] + (-27.2)y[n-2] + (53.91)y[n-3] + (-67.02)y[n-4] + (53.51)y[n-5] + (-26.8)y[n-6] + (7.697)y[n-7] + (-0.9707)y[n-8] + 1.039337e-$
 Persamaan LCCDE Filter Frekuensi Harmonik 2
 $y(n) = (7.746)y[n-1] + (-26.47)y[n-2] + (52.12)y[n-3] + (-64.65)y[n-4] + (51.73)y[n-5] + (-26.08)y[n-6] + (7.575)y[n-7] + (-0.9707)y[n-8] + 1.039337$

Representasi Sistem Filter Cheby1 Built-In
 Persamaan LCCDE Filter Frekuensi Fundamental
 $y(n) = (7.961)y[n-1] + (-27.75)y[n-2] + (55.34)y[n-3] + (-69.03)y[n-4] + (55.15)y[n-5] + (-27.57)y[n-6] + (7.88)y[n-7] + (-0.9864)y[n-8] + 3.749242e$
 Persamaan LCCDE Filter Frekuensi Harmonik 1
 $y(n) = (7.886)y[n-1] + (-27.31)y[n-2] + (54.23)y[n-3] + (-67.56)y[n-4] + (54.05)y[n-5] + (-27.12)y[n-6] + (7.806)y[n-7] + (-0.9864)y[n-8] + 3.749242$
 Persamaan LCCDE Filter Frekuensi Harmonik 2
 $y(n) = (7.762)y[n-1] + (-26.58)y[n-2] + (52.44)y[n-3] + (-65.17)y[n-4] + (52.26)y[n-5] + (-26.4)y[n-6] + (7.683)y[n-7] + (-0.9864)y[n-8] + 3.749242e$

Representasi Sistem Filter FIR1 Built-In
 Persamaan LCCDE Filter Frekuensi Fundamental
 $y(n) = - (1.177e-05)x[n] - (9.171e-06)x[n-1] - (6.394e-06)x[n-2] - (3.455e-06)x[n-3] - (3.681e-07)x[n-4] + (2.85e-06)x[n-5] + (6.181e-06)x[n-6] + (9$
 Persamaan LCCDE Filter Frekuensi Harmonik 1
 $y(n) = - (2.922e-05)x[n] - (3.306e-05)x[n-1] - (3.619e-05)x[n-2] - (3.851e-05)x[n-3] - (3.992e-05)x[n-4] - (4.035e-05)x[n-5] - (3.974e-05)x[n-6] - ($
 Persamaan LCCDE Filter Frekuensi Harmonik 2
 $y(n) = (3.049e-05)x[n] + (2.533e-05)x[n-1] + (1.847e-05)x[n-2] + (1.026e-05)x[n-3] + (1.11e-06)x[n-4] - (8.487e-06)x[n-5] - (1.799e-05)x[n-6] - (2.6$

Representasi Sistem Filter Peletakkan Manual Pole-Zero Sederhana
 Persamaan LCCDE Filter Frekuensi Fundamental
 $y(n) = (5.947)y[n-1] - (14.76)y[n-2] + (19.55)y[n-3] - (14.59)y[n-4] + (5.813)y[n-5] - (0.9663)y[n-6] + (1)x[n] - (3)x[n-2] + (3)x[n-4] - (1)x[n-6]$
 Persamaan LCCDE Filter Frekuensi Harmonik 1
 $y(n) = (5.891)y[n-1] - (14.53)y[n-2] + (19.22)y[n-3] - (14.37)y[n-4] + (5.758)y[n-5] - (0.9663)y[n-6] + (1)x[n] - (3)x[n-2] + (3)x[n-4] - (1)x[n-6]$
 Persamaan LCCDE Filter Frekuensi Harmonik 2
 $y(n) = (5.798)y[n-1] - (14.17)y[n-2] + (18.68)y[n-3] - (14.01)y[n-4] + (5.667)y[n-5] - (0.9663)y[n-6] + (1)x[n] - (3)x[n-2] + (3)x[n-4] - (1)x[n-6]$

Gambar 38 - Contoh Representasi Sistem Filter Sinyal “toooot” dengan LCCDE

Rancangan filter yang digunakan untuk memisahkan keluaran sinyal “teeet.. ..teeet.. ..teeet.. ...” dan sinyal “toooot.. ..toooot.. ..toooot.. ...” adalah *Bandpass Filter*. Filter ini digunakan untuk meloloskan frekuensi fundamental dan frekuensi harmonik sekaligus membuang frekuensi lain yang berpotensi menyebabkan *noise*. BPF dirancang dengan menetapkan frekuensi cut-off low dan frekuensi cut-off high disekitar frekuensi yang menjadi target, hal ini didapat dengan menentukan lebar *bandwidth* sebesar 80 Hz agar jarak pole cukup aman dari lingkaran satuan, *bandwidth* yang terlalu sempit akan

membuat sistem menjadi tidak stabil dan sangat sensitif karena pole pada filter akan diletakkan dekat dengan lingkaran satuan. Misalnya, pada frekuensi fundamental sinyal “teeet” sebesar 758.372 Hz akan memiliki $f_c(low)$ sebesar 718.372 Hz dan $f_c(high)$ sebesar 798.372 Hz, sedangkan pada frekuensi fundamental sinyal “tooot” sebesar 555.826 Hz akan memiliki $f_c(low)$ sebesar 515.826 Hz dan $f_c(high)$ sebesar 595.826 Hz.

Pemilihan orde untuk rancangan filter ini didasarkan pada pemfokusan terhadap sinyal “teeet” dan “tooot”. IIR menggunakan orde 8 untuk mendapatkan *roll-off* yang cukup tajam untuk meredam suara di luar frekuensi target dan menghilangkan noise. Orde ini terbilang cukup efisien secara komputasi dan tidak terlalu tinggi sehingga dapat meminimalisir ketidakstabilan numerik. Sedangkan, orde 1000 digunakan pada FIR untuk dapat mencapai kinerja pemotongan frekuensi yang setara dengan IIR. Berdasarkan Gambar 31 dan Gambar 35, terlihat respons impuls sekitar 600 sampel, hal ini menunjukkan FIR membutuhkan orde yang cukup tinggi untuk dapat memenuhi spesifikasi *cut-off* yang tajam dan *bandwidth* yang sempit.

Pada analisis perancangan filter untuk memisahkan sinyal “teeet” dan “tooot” dilakukan perbandingan antara beberapa jenis filter digital, filter *Butterworth IIR*, *Chebyshev IIR I*, dan *FIR based Windowed*. Filter-filter tersebut dirancang menggunakan *built-in function* yang menyediakan berbagai *computational task*. Selain itu, karakteristik desain filter ini juga dibandingkan secara konseptual dengan menggunakan pendekatan penempatan pole-zero secara manual. Perbandingan filter dilakukan melalui beberapa representasi sistem, yaitu respons impuls, peta pole-zero, LCCDE, respons magnitudo, dan respons fasa.

Berdasarkan Gambar 29 dan Gambar 33 yang menunjukkan hasil filter dengan *Butterworth* untuk sinyal “teeet” dan “tooot”, terlihat bahwa respons impuls berosilasi dan menurun secara eksponensial. Hal ini menunjukkan karakteristik IIR yang berdurasi tak hingga. Respons magnitudo menunjukkan karakteristik yang *maximally flat* tanpa ripple dengan puncak yang tajam pada frekuensi yang menjadi target, sedangkan respon fasa memiliki sifat non-linear yang menjadi ciri umum IIR. Peta pole-zero memperlihatkan bahwa pole-pole sistem terletak di dalam lingkaran satuan dan relatif mendekati lingkaran satuan, hal tersebut akan membentuk sistem yang stabil dan transisi antar frekuensi target menjadi halus.

Selanjutnya, Gambar 30 dan Gambar 34 yang menunjukkan hasil filter dengan *Chebyshev I* untuk sinyal “teeet” dan “tooot”. Respons impuls filter *chebyshev* menunjukkan IIR dengan osilasi yang lebih kuat dibandingkan *butterworth*, terlihat dari bagian awal respons yang frekuensi fundamentalnya memiliki lebih banyak sampel. Hal ini berkaitan dengan adanya *ripple* di passband seperti pada gambar respon magnitudo. Filter *Chebyshev* memiliki transisi yang lebih tajam sehingga dapat memisahkan komponen fundamental dan harmonik dengan lebih selektif. Peta pole-zero menunjukkan pole yang dekat dengan lingkaran satuan yang menyebabkan peningkatan kecuraman respons frekuensi., sedangkan respons fasa filter *chebyshev* juga memiliki bentuk yang linear.

Hasil perancangan dengan menggunakan *FIR based windowed* ditunjukkan pada Gambar 31 dan Gambar 35, terlihat respons impuls filter FIR bersifat simetris, memiliki durasi yang terbatas dan menghasilkan respons fasa yang linear sehingga tidak menyebabkan distorsi gelombang pada domain waktu. Peta pole-zero menunjukkan bahwa seluruh pole berada di titik origin, sedangkan zero-zero filter tersebar di sekitar lingkaran satuan membentuk cincin. Dengan konfigurasi seperti ini akan membentuk sistem yang stabil. Respons magnitudo FIR menunjukkan passband dengan terdapat sedikit *ripple* dan *side-lobe* yang disebabkan karena *windowing*.

Hasil filter dengan peletakkan pole-zero secara manual dapat dilihat pada Gambar 32 dan Gambar 36. Untuk merancang filter digunakan orde 9, dengan tujuan terbentuknya atenuasi yang curam sehingga respons frekuensi dapat spesifik. Pada pembuatannya zero diletakkan tepat di lingkaran satuan, dimana

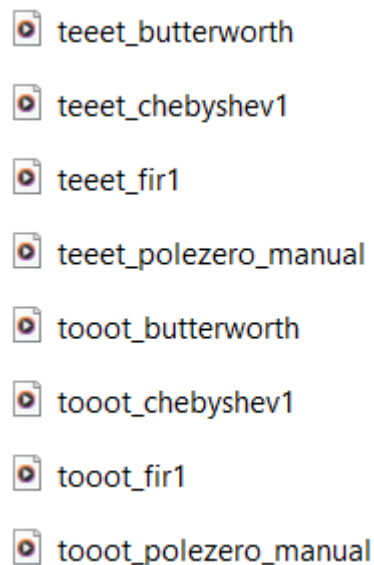
$w=0$ dan $w=\pi$, jika digambarkan letaknya di 1 dan -1 sumbu real. Sesuai dengan bandwidth yang digunakan, maka nilai r yang digunakan adalah sekitar 0.99. Efek dari kedua hal tersebut adalah terciptanya resonansi yang dapat mengangkat amplitudo sinyal frekuensi target dan teredamnya suara-suara yang dapat menjadi gangguan. Namun, jika dibandingkan dengan penggunaan filter standar lainnya, filter hasil desain algoritma (Butterworth, Chebyshev, FIR) tetap dianggap lebih bagus secara teknis. Hal ini disebabkan karena penempatan pole-zero yang dilakukan berdasarkan intuisi visual. Akibatnya, bentuk kurva respons frekuensi di dalam passband seringkali tidak rata. Terlihat dari gambar hasil respon magnitudo, tingkat kecuraman puncak yang dihasilkan tidak setajam hasil *butterworth* yang cukup datar dan *chebyshev* dengan kecuraman dengan transisi yang cukup cepat.

Analisis persamaan LCCDE untuk masing-masing filter terdapat pada Gambar 37 untuk sinyal “teeet” dan Gambar 38 untuk sinyal “tooot”. Analisis filter IIR memiliki bentuk rekursif yang membutuhkan *feedback* dari masa lalu. Terlihat dari hasil *butterworth* dan *chebyshev* terdapat komponen delay di $y(n-8)$, hal ini menunjukkan kedua filter dirancang dengan menggunakan orde 8. Sedangkan, analisis filter FIR yang dikenal juga dengan filter non-rekursif yang persamaannya berisikan komponen input $x(n)$ saja untuk menghasilkan output. FIR memiliki sistem BIBO atau selalu stabil karena tidak ada penyebut di dalam fungsi transfer atau semua pole berada di origin sehingga respons fasanya linear.

d. membuat program Matlab yang menghitung 2 sinyal keluaran filter yang dirancang dan menyimpan dalam format *.mp3.

Jawab:

Pembuatan file audio sinyal keluaran filter dapat dilaksanakan dengan bantuan kode Matlab [3] dan jika kode tersebut dijalankan, file audio sinyal keluaran filter yang dihasilkan adalah sebagai berikut,



Gambar 39 - File-File Audio mp3 Sinyal “teeet” dan Sinyal “tooot” Hasil Filtering

Audio input didefinisikan sebagai x sebelumnya, dan dengan desain kode berikut ini akan dipisahkan komponen suara “teeet” dan “tooot” yang kemudian akan disimpan dalam bentuk mp3 untuk masing-masing komponen suara.

Untuk pemrosesan komponen suara “teeet”, audio input difilter dengan menggunakan tiga sub-filter di tiap masing-masing jenis filter. Hal ini dikarenakan komponen sinyal suara “teeet” tersusun atas tiga komponen frekuensi dominan yang kemudian akan diloloskan oleh masing-masing sub-filter. Tiga filter yang digunakan adalah Butterworth, Chebyshev Tipe 1, dan FIR.

1. Filter Butterworth merupakan filter IIR yang memiliki karakteristik *maximally flat* pada *passband*-nya, yaitu tidak ada *ripple* di area yang diloloskan, namun transisi antara *passband* dan *stopband*nya relatif landai.
2. Filter Chebyshev Tipe 1 yang digunakan merupakan filter IIR dengan *ripple* di *passband* namun memiliki transisi yang lebih tajam dibanding Butterworth pada orde yang sama.
3. Filter FIR yang didesain dengan *windowing* akan memiliki fasa linear untuk menjaga gelombang dari sinyal audio input.
4. Filter Pole-Zero Manual, merupakan filter pole dan zero yang didesain dan didefinisikan secara manual dimana koefisien (b,a) ditentukan secara eksplisit memberikan kontrol atas respons frekuensi.

Masing-masing filter ini ditargetkan pada komponen frekuensi harmonik spesifik, pada kode ini sebanyak 3 frekuensi harmonik untuk komponen “teeet”. Frekuensi harmonik spesifik didefinisikan dengan memberikan batas atas dan bawah dari frekuensi harmonik yang ditargetkan.

Output dari masing-masing subfilter di tiap jenis filter dijumlahkan untuk merekonstruksi sinyal “teeet” utuh yang telah melalui proses penyaringan sesuai dengan jenis filternya. Sehingga subfilter ini memiliki sifat yang komplementer, dimana menangkap pita frekuensi berbeda namun setelah dijumlahkan membentuk spektrum suara “teeet”. Kemudian sinyal hasil rekonstruksi tiap filter dinormalisasi amplitudonya untuk mencegah adanya *clipping* saat pemutaran audio dan memastikan perbandingan antar file audio berbasis kualitas filter, bukan perbedaan volume audio. Setelah pemrosesan sinyal audio dengan filter, empat versi sinyal “teeet” dari masing-masing filter (Butterworth, Chebyshev Tipe 1, FIR, Pole-Zero Manual) disimpan sebagai file MP3. Frekuensi sampling berperan untuk menentukan laju sampling audio output. Untuk mengonfirmasi proses penyimpanan, ditampilkan pesan konfirmasi output tersimpan setelah file tersimpan.

Untuk pemrosesan sinyal “tooot” secara struktur tahapan pemfilteran identik dengan yang digunakan untuk pemrosesan sinyal “teeet”, perbedaan muncul untuk koefisien filter yang didefinisikan secara khusus untuk karakteristik spektral suara “tooot” Frekuensi dominan sinyal “tooot” umumnya lebih rendah dan durasi sinyalnya lebih panjang, sehingga koefisien masing-masing parameter perlu didefinisikan ulang secara spesifik untuk komponen sinyal “tooot”.

Secara strukturnya kode yang digunakan dalam soal 1D ini akan menghasilkan output yang dapat membandingkan kualitas audio dari hasil pemfilteran empat pendekatan desain filter yang berbeda. Selain itu menunjukkan komponen frekuensi suara dapat direkonstruksi melalui bank filter berbasis sub-band. Sehingga diperoleh sebuah dataset audio hasil filter yang dapat dievaluasi kualitas untuk masing-masing jenisnya.

Filter Butterworth yang digunakan dalam pemrosesan suara “teeet” dan “tooot” memiliki fungsi untuk memisahkan pita frekuensi secara selektif dan *smooth*. Karena tidak ada *ripple* di *passband*, dan amplitudo pada frekuensi yang ditargetkan tetap konsisten tanpa modulasi yang disebabkan oleh osilasi respons filter. Sehingga sinyal audio yang dihasilkan integritas nada dan kejernihan pendengarannya terjaga. Namun, transisi antara *passband* dan *stopband* relatif landai dibandingkan filter IIR lainnya di orde yang sama. Frekuensi yang berada dekat dengan batas *cut-off* mungkin tidak sepenuhnya ditekan, sehingga ada frekuensi “tidak diinginkan” yang berdekatan masih mungkin lolos dalam jumlah kecil. Suara yang dihasilkan cenderung bersih, stabil, dan bebas dari distorsi nada, tidak muncul suara dengung yang muncul akibat resonansi berlebihan atau respons *ripple*. Dapat disimpulkan bahwa filter Butterworth menawarkan hasil keluaran yang mengombinasikan antara kehalusan respons dan stabilitas suara dan menjaga audio yang natural dibandingkan selektivitas ekstrem.

Filter Chebyshev yang digunakan dalam pemrosesan suara “teeet” dan “tooot” memiliki fungsi untuk meloloskan frekuensi tertentu secara spesifik dengan transisi antar frekuensi yang lebih tajam, sehingga

frekuensi di luar pita yang ditargetkan akan ditekan dengan jauh lebih efektif. Namun, keberadaan *ripple passband* dapat menyebabkan modulasi amplitudo ringan pada komponen frekuensi di pita yang diloloskan, efek dari *ripple* ini tidak terlalu mengganggu selama terkontrol di bawah 0,5 - 1 dB karena nyaris tidak terdengar oleh telinga manusia. Output audio yang dihasilkan oleh filter ini cenderung lebih tajam dan terisolasi pada sinyal audio spesifik yang ditargetkan, latar audio akan tampak lebih sunyi. Tidak terdengar distorsi harmonik atau efek samping yang mengganggu. Sehingga dapat disimpulkan bahwa Filter Chebyshev Tipe 1 memiliki keunggulan dalam efisiensi peredaman dan cocok untuk aplikasi yang membutuhkan penekanan interferensi di luar pita yang dipilih sebagai prioritas utama meskipun transisinya lebih tajam.

Filter FIR yang digunakan dalam pemrosesan suara “teeet” dan “tooot” umumnya menggunakan metode windowing untuk mendekati respons frekuensi ideal. Filter ini memiliki keunggulan untuk mencapai fase linier sempurna jika koefisiennya simetris. Fase linier yang simetris berarti memiliki delay grup konstan di seluruh pita frekuensi, sehingga semua komponen frekuensi mengalami penundaan waktu yang sama. Untuk sinyal suara transien yang cenderung cepat seperti “teeet”, fase linear penting untuk menjaga distorsi dalam bentuk gelombang atau *smearing*. Filter FIR menjaga bentuk suara yang lebih utuh. Namun, kelemahannya dibutuhkan filter FIR berorde tinggi dibanding IIR untuk mencapai transisi selebar serupa. Komputasi yang semakin banyak untuk mengimplementasikan FIR orde tinggi menyebabkan kemungkinan adanya delay latensi. Dampaknya pada sinyal audio output keluaran suara yang dihasilkan tajam dan jelas tanpa adanya efek gangguan tipis yang umumnya muncul pada filter IIR. Sehingga suaranya tampak lebih *real* dibandingkan dua filter sebelumnya. Dapat disimpulkan filter FIR memberikan preservasi bentuk gelombang dan *onset time* yang baik meskipun lebih kompleks pada sisi komputasinya.

Untuk desain manual pole-zero yang merupakan representasi sistem dalam domain-z. Fungsi pole sendiri untuk menciptakan resonansi, semakin dekat pole dengan lingkaran satuan maka resonansi yang diciptakan semakin tajam. Sementara zero akan menekan frekuensi tersebut. Pendekatan ini cukup fleksibel karena pole dan zero dapat diletakkan secara strategis sesuai kebutuhan, untuk implementasi pada sinyal input audio ini idealnya ditempatkan tiga pasang pole kompleks konjugat di sekitar frekuensi yang ditargetkan. Keberhasilan desain ini ditentukan pada jumlah pole, jarak pole ke lingkaran satuan, dan penempatan zero. Berdasarkan desain yang diimplementasikan jumlah pole masih terlalu sedikit atau terlalu jauh dari lingkaran satuan sehingga resonansinya cenderung lemah, dan orde sistem masih terlalu rendah untuk mencapai selektivitas yang diinginkan, sehingga output yang dihasilkan masih sedikit kurang jernih dibandingkan filter lainnya. Untuk desain pole zero dapat diperbaiki dengan meningkatkan order sistem, mendekatkan pole ke lingkaran satuan, dan menambahkan zero untuk menekan komponen DC. Apabila desain diperbaiki maka desain pole-zero manual memiliki potensi tertinggi untuk dirancang sesuai kebutuhan output yang diinginkan.

Pelajari *built-in functions* berikut:

`fvtool, spectrogram, fft, abs, angle, log10, butter, cheby1, fir1, freqz, tf2zpk, zp2tf, zplane, filter, conv`

Nilai: 30

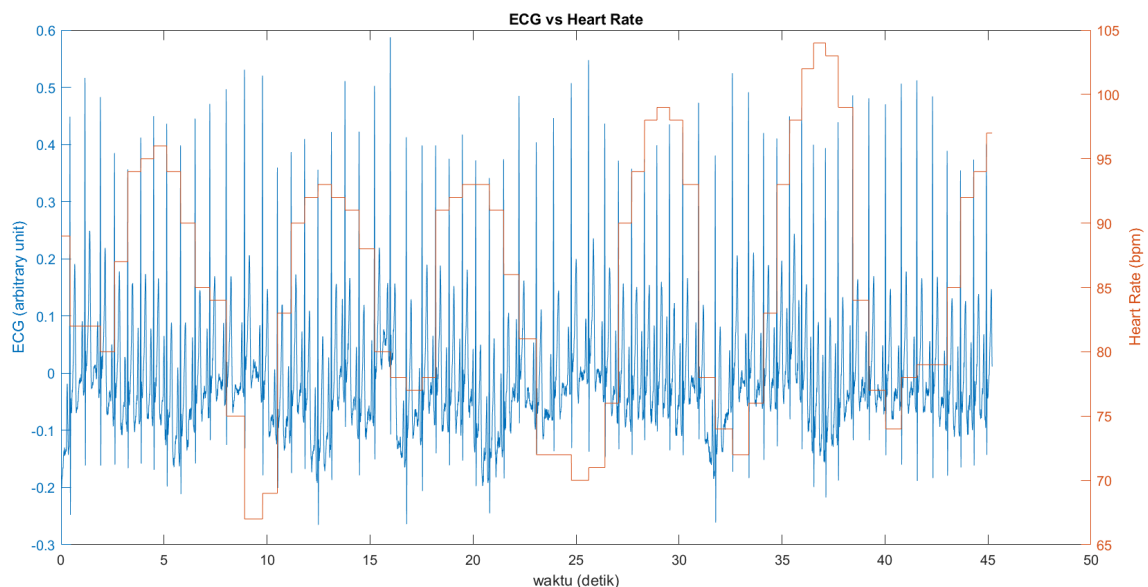
a	b	c	d
2	12	13	3

Soal 2: ECG

Diberikan sinyal ECG hasil perekaman Biopac:

Nama file	Parameter pengukuran
Dataset_EKG_HR_1000Hz.csv	- 1 buah sinyal ECG single channel dalam arbitrary unit; - nilai rujukan Heart Rate (HR) dalam integer bpm; - ADC: 1000 sampel/detik;
BlandAltman.m	Fungsi untuk menampilkan korelasi
Matlab soal 2	Tulis code di file ini

Kedua sinyal tersebut ditampilkan pada Gambar 1. Pada skenario ini, terlihat bahwa nilai HR berubah tergantung subjek dalam keadaan inhale atau exhale. Nilai rujukan HR tersebut dihitung dari durasi segmen RR di domain waktu. Dengan kata lain, dalam 1 periode sinyal ECG, diasumsikan nilai HR konstan. Metode perhitungan di domain waktu ini banyak dipraktikkan karena kemudahan deteksi puncak R yang memiliki amplitudo relatif paling tinggi dibandingkan puncak lainnya. Untuk memperbanyak nilai HR, bisa dilakukan juga deteksi setiap puncak P, Q, R, S, dan T, sehingga akan terdapat 5 nilai HR dalam 1 periode sinyal ECG, yakni durasi segmen PP, QQ, RR, SS, dan TT. Dalam tubes ini, tidak hanya 5 nilai HR, tetapi Anda diminta untuk menghitung nilai HR untuk setiap sampel ECG. Lakukan perhitungan nilai HR ini di domain frekuensi. Disepakati bahwa nilai HR adalah integer.



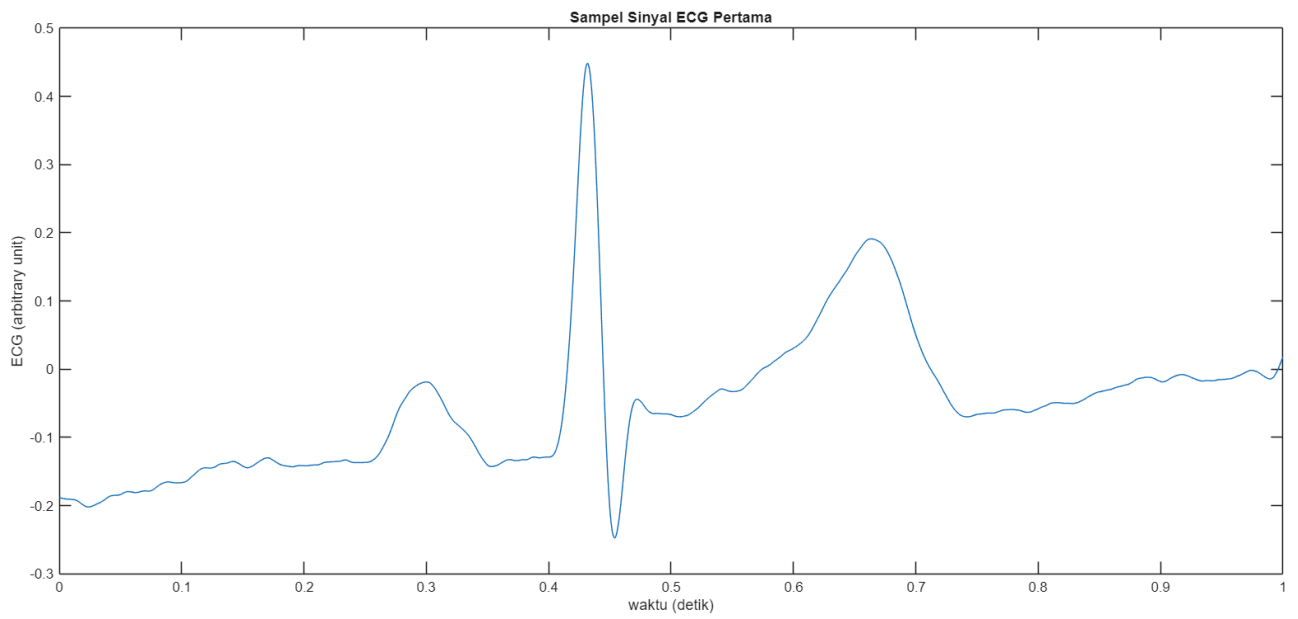
Gambar 1. Sinyal ECG vs Heart Rate

a. menghitung dan menampilkan *power spectrum density* (PSD) sinyal atau *spectrogram*.

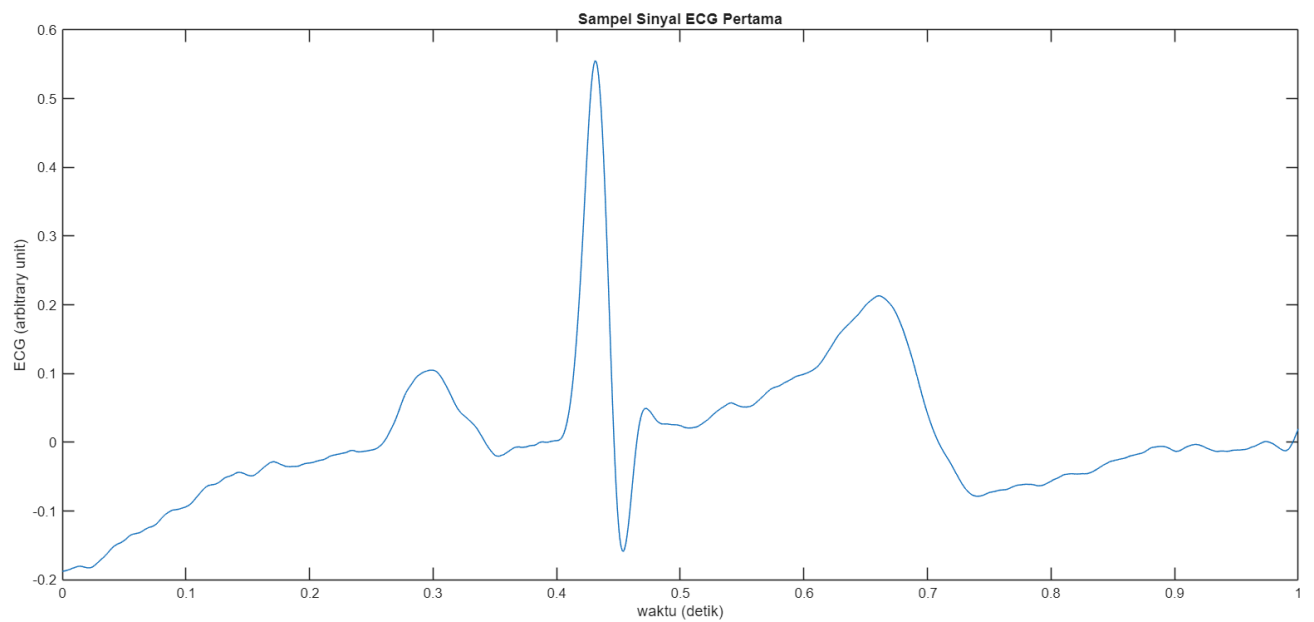
Tips: lakukan seperti soal 1

Jawab:

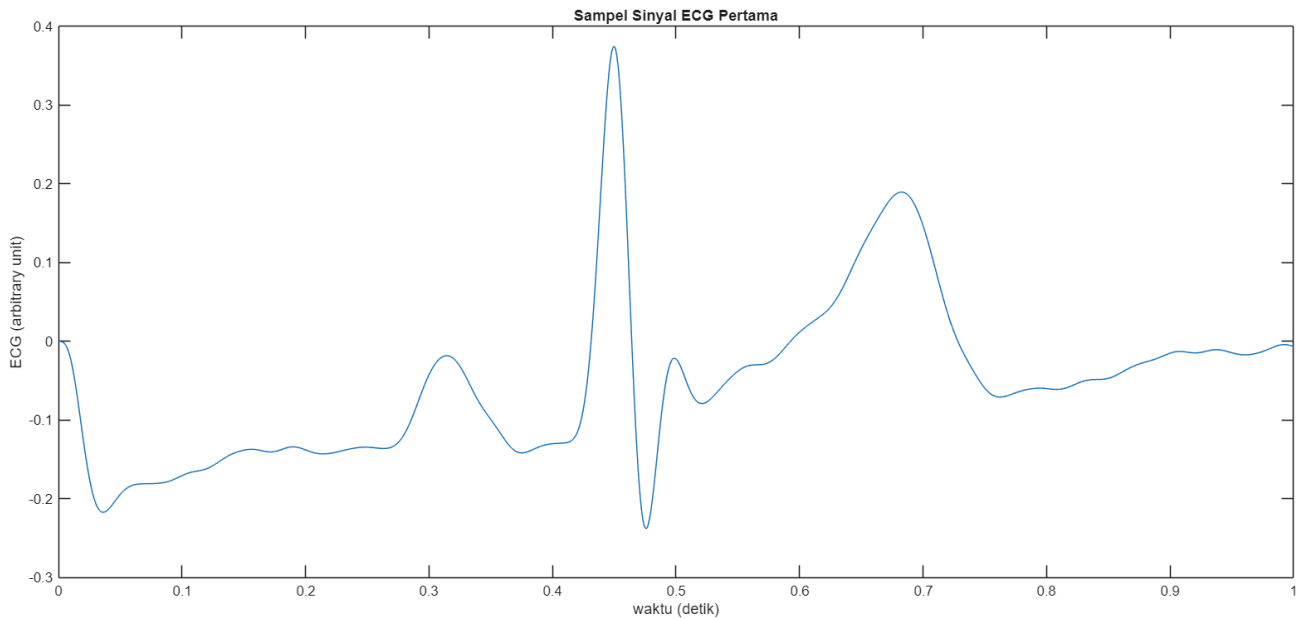
Sebelum analisis PSD dilakukan, sinyal ECG difilter terlebih dahulu untuk menghilangkan *noise* yang tidak diinginkan. Terdapat 2 filter yang dapat digunakan untuk menghilangkan *noise* sinyal ECG, yaitu *high-pass filter* (HPF) dan *low-pass filter* (LPF). Berikut adalah dampak dari setiap filter terhadap sampel sinyal ECG yang pertama,



Gambar 39 - Sampel Sinyal ECG Pertama



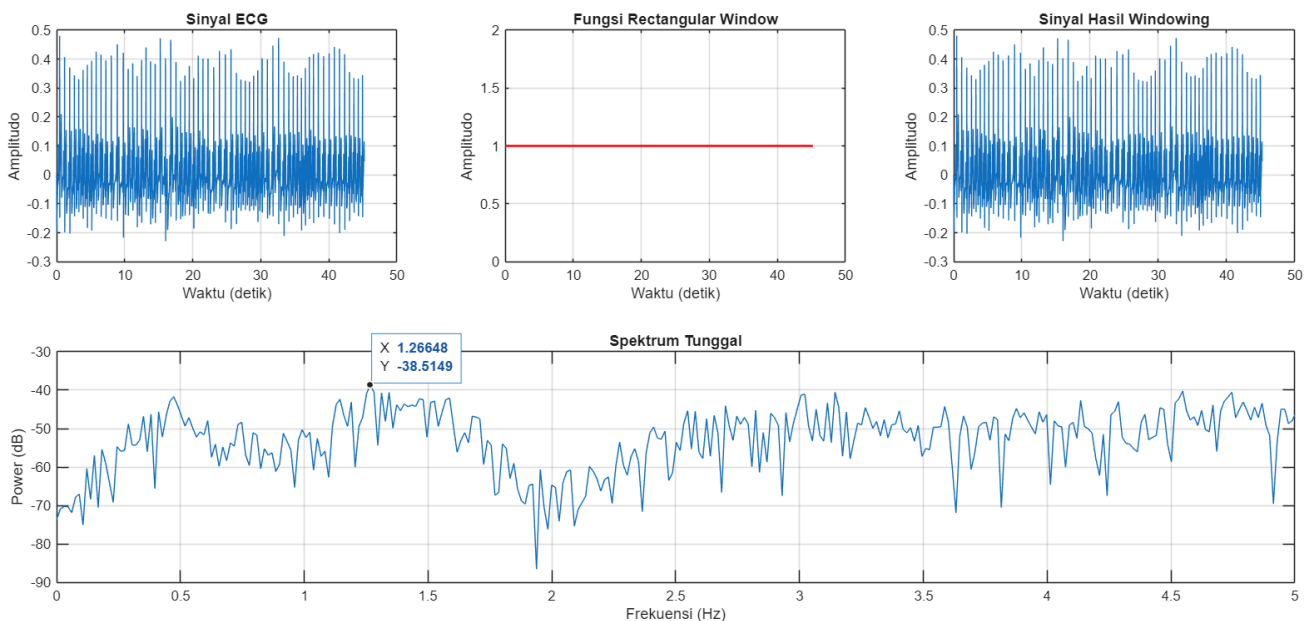
Gambar 40 - Sampel Sinyal ECG Pertama Setelah Diterapkan HPF



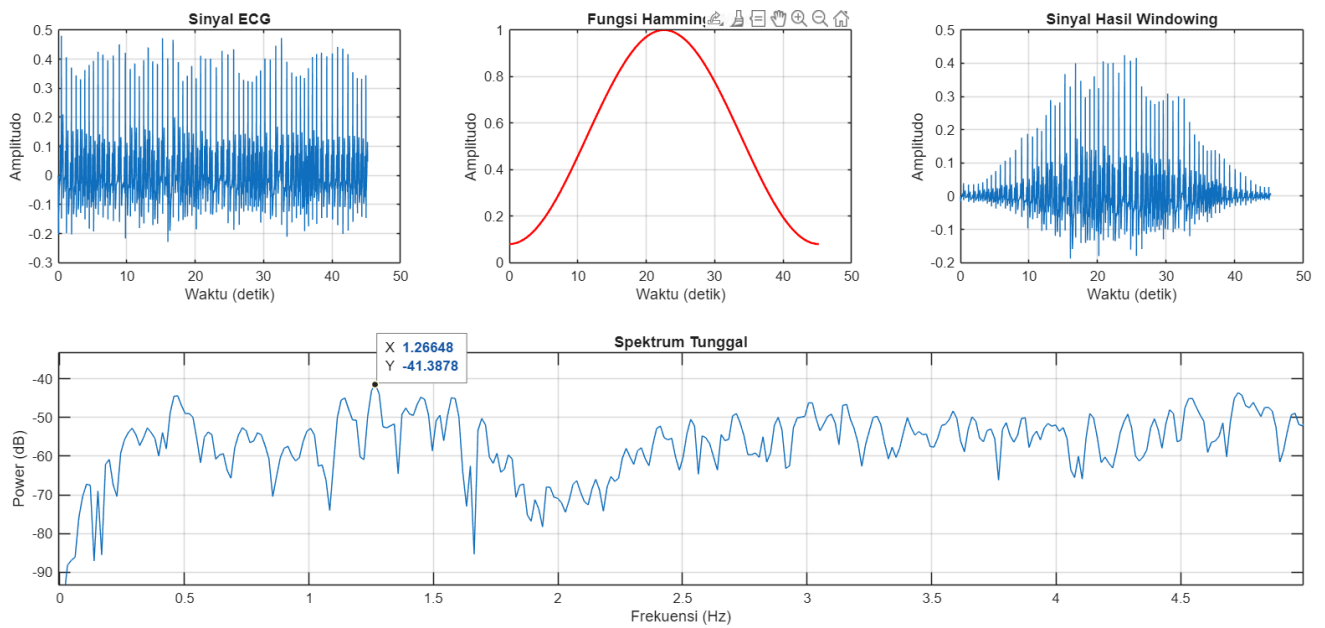
Gambar 41 - Sampel Sinyal ECG Pertama Setelah Diterapkan LPF

Berdasarkan gambar-gambar sampel sinyal ECG pertama setelah diterapkan filter tersebut, terlihat bahwa HPF Butterworth orde 2 dengan frekuensi *cut-off* 0,5 Hz berhasil meredam *baseline wander* yang terdapat pada sinyal ECG mentah dan mempertahankan morfologi PQRST pada sinyal ECG. Setelah itu, LPF Butterworth orde 4 dengan frekuensi *cut-off* 25 Hz berhasil diimplementasikan tanpa mengubah morfologi PQRST pada sinyal ECG sehingga *noise* frekuensi tinggi dapat diredam tanpa menghilangkan informasi sinyal ECG. Berdasarkan validasi tersebut bahwa HPF dan LPF tersebut telah dapat memfilter sinyal ECG sesuai dengan harapan, filter yang diterapkan kepada sinyal ECG merupakan HPF dan LPF yang disusun secara kaskade.

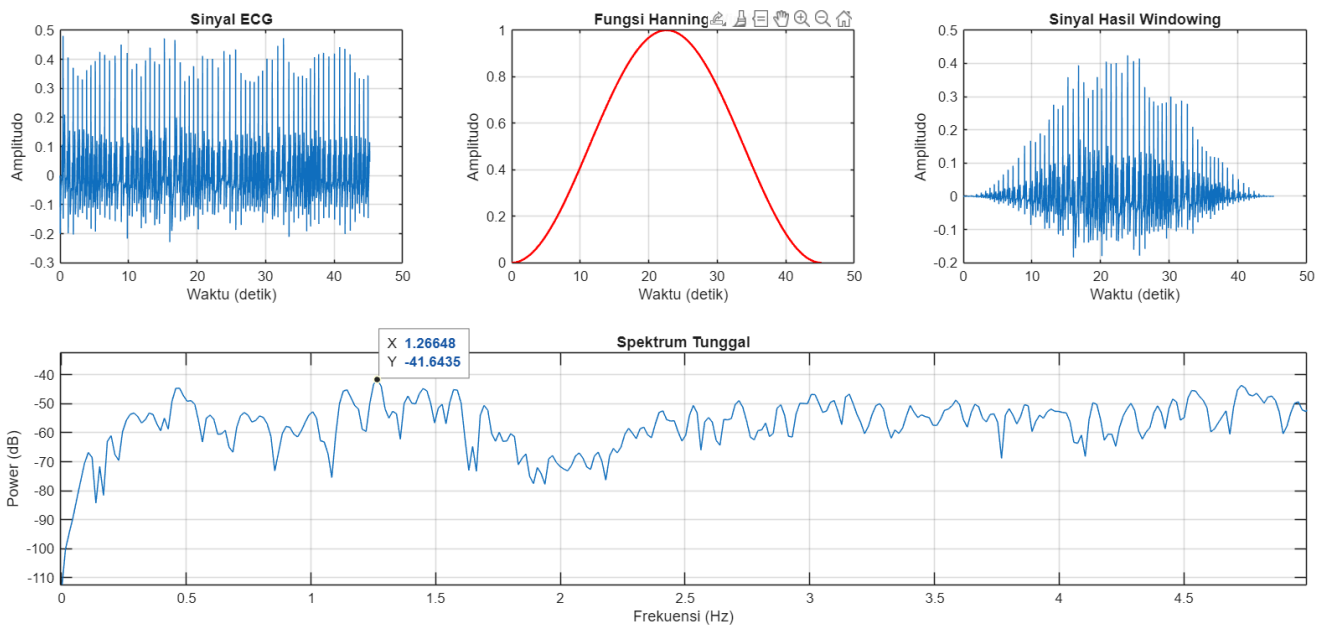
Pembuatan PSD dapat dilaksanakan dengan bantuan kode Matlab [3] dan jika kode tersebut dijalankan, PSD yang diperoleh adalah sebagai berikut,



Gambar 42 - Analisis PSD Sinyal ECG dengan Rectangular Window



Gambar 43 - Analisis PSD Sinyal ECG dengan Hamming Window



Gambar 44 - Analisis PSD Sinyal ECG dengan Hanning Window

Berdasarkan hasil analisis PSD tersebut, frekuensi puncak dari sinyal ECG tersebut berada pada 1,26648 Hz sehingga HR rata-rata dari seluruh sampel sinyal ECG tersebut dapat diperkirakan bernilai sebagai berikut,

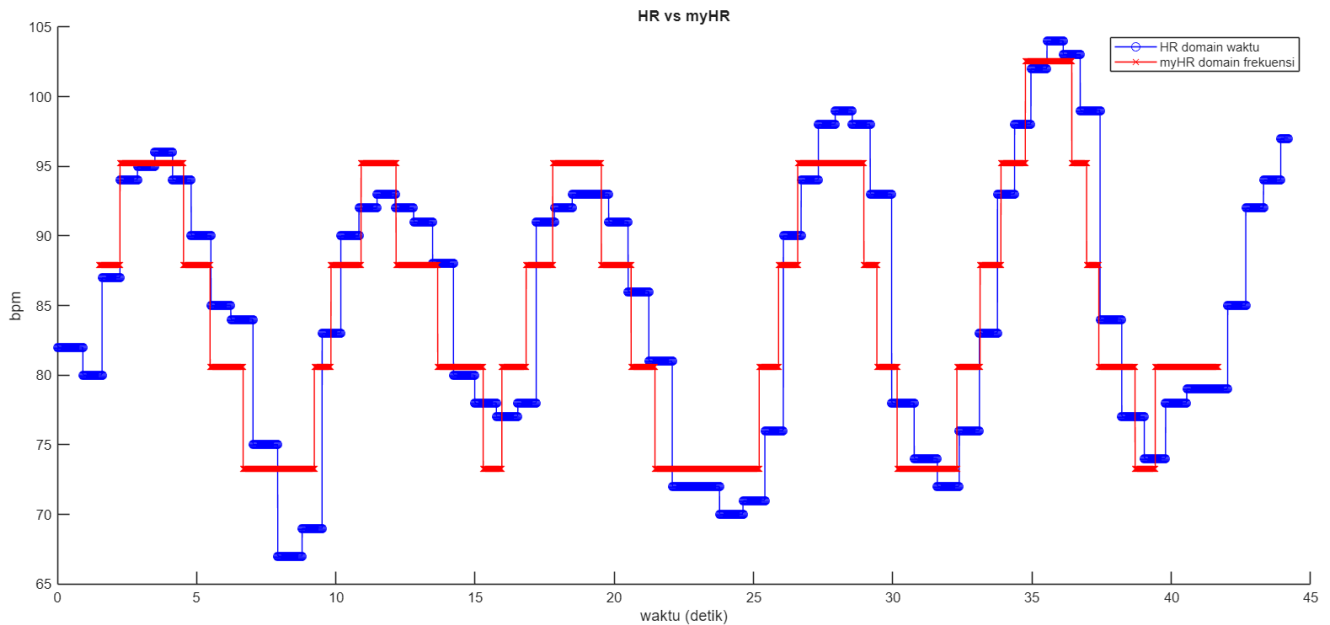
$$HR_{rata-rata} = f_{puncak} \times 60$$

$$HR_{rata-rata} = 1,26648 \times 60 = 75,9888 \text{ BPM}$$

b. menghitung nilai myHR untuk setiap sampel sinyal ECG di domain frekuensi dan bandingkan dengan referensi HR.

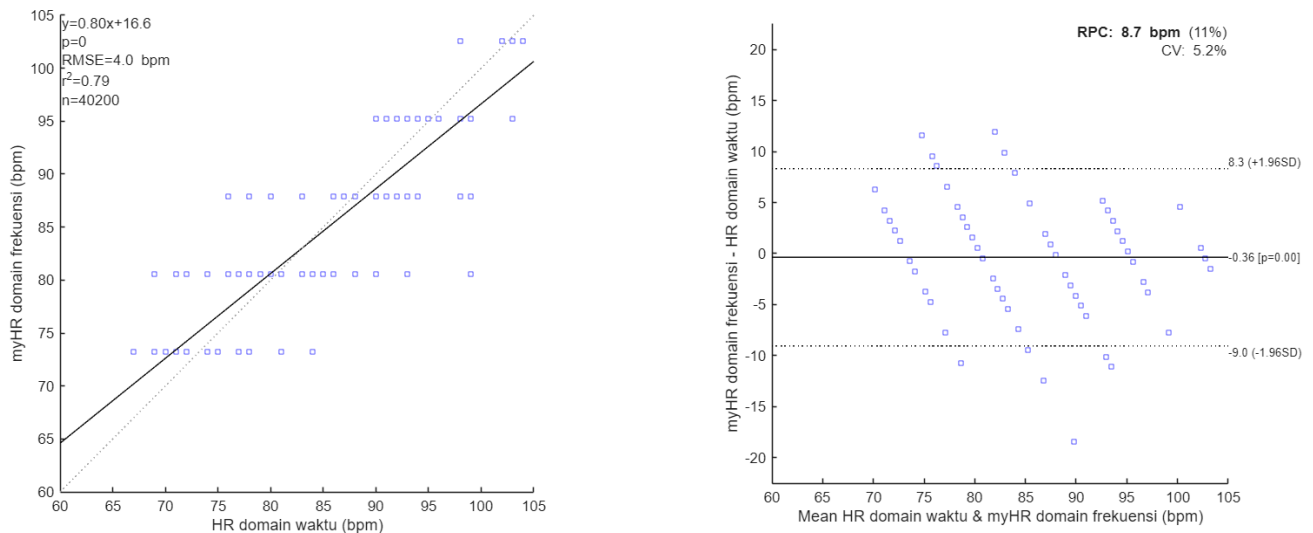
Jawab:

Perhitungan nilai myHR untuk setiap sampel sinyal ECG di domain frekuensi dapat diimplementasikan dengan *sliding window*, yaitu teknik untuk menghitung perubahan fitur secara dinamis terhadap waktu, seperti nilai HR, dengan cara mengambil potongan data yang disebut *window* dan menggesernya sepanjang sinyal utama. Teknik tersebut dapat dilaksanakan dengan bantuan kode Matlab [3] dan jika kode tersebut dijalankan, perbandingan nilai HR referensi dan myHR, serta korelasi antara keduanya adalah sebagai berikut,



Gambar 45 - Perbandingan Grafik Nilai HR Referensi dan myHR

Correlation and Bland-Altman Plots



Gambar 46 - Korelasi dan Plot Bland-Altman HR Referensi dan myHR

Berdasarkan hasil perbandingan tersebut, terlihat bahwa nilai myHR dan HR berada pada kisaran nilai yang sama, tetapi tidak sepenuhnya sama dengan nilai korelasi antara keduanya (r^2) adalah 0,79. Perbedaan tersebut dapat disebabkan oleh beberapa hal, yaitu sebagai berikut,

1. Perbedaan sifat teknik estimasi nilai HR, yaitu teknik pengukuran HR referensi yang menggunakan pengukuran durasi segmen RR secara instan yang dapat menentukan waktu kejadian tiap detak jantung secara akurat, tetapi pengukuran menjadi sensitif terhadap *noise*, sedangkan *sliding window* mengambil rata-rata energi detak jantung dalam satu jendela waktu sehingga hasil HR lebih stabil terhadap *noise*, tetapi sulit untuk menangkap nilai HR yang berubah secara instan.
2. Keterbatasan resolusi frekuensi pada pengukuran myHR dengan *sliding window* karena nilai sampel segmentasi (N2) yang digunakan dalam *Fast Fourier Transform* (FFT).
3. Nilai HR referensi belum tentu benar secara mutlak karena nilai HR referensi dapat dipengaruhi tingkat keakuratannya oleh ketidakteitian deteksi puncak R dan artefak gerak.

Pelajari *built-in functions* berikut:

`fvtool, spectrogram, fft, abs, angle, log10, butter, cheby1, fir1, freqz, tf2zpk, zp2tf, zplane, filter, conv`

Nilai: 30

a	b
5	25

Soal 3: Real-time processing

Diberikan 2 files pada folder Matlab soal 3:

Nama file	Parameter pengukuran
noise.wav	- 1 buah sinyal audio noise;
Matlab soal 3.m	Tulis code merujuk pada file ini

, Anda diminta untuk mengurangi efek noise pada suara manusia secara real-time, seperti diilustrasikan pada Gambar 2. Tampak seperti bercanda, tetapi permasalahan seperti ini terjadi pada komunikasi pilot-kopilot di helikopter. Untuk menjalankan skenario ini, dibutuhkan:

- 1) Ear plugs (penyumbat telinga);
- 2) HP;
- 3) Berbagai macam Matlab toolbox (library), e.g. audio toolbox dan dsp toolbox. Silakan install sesuai permintaan ketika menjalankan file Matlab_soal_3.m.

Skenario ini bisa juga dilakukan dengan mengganti Stevanie dengan sebuah HP lainnya yang membunyikan file suara ucapan manusia dengan tujuan agar Stevani tidak stress mendengar suara “ngiiing”. Namun, pastikan suara yang dibunyikan adalah tetap suara ucapan manusia, bukan suara lumba-lumba atau kelelawar atau instrumen musik.

Kumpulkan juga dalam folder yang sama:

- 1) Satu buah file audio: mySpeechInput.wav, yaitu sinyal suara terkontaminasi noise;
- 2) Satu buah file audio: mySpeechOutput.wav, yaitu sinyal suara tersisa, di mana noise telah dikurangi.
- 3) Satu buah file Matlab jawaban 3.a: Matlab_soal_3a.m
- 4) Satu buah file Matlab jawaban 3.b: Matlab_soal_3b.m
- 5) Satu buah file Matlab jawaban 3.c: Matlab_soal_3c.m

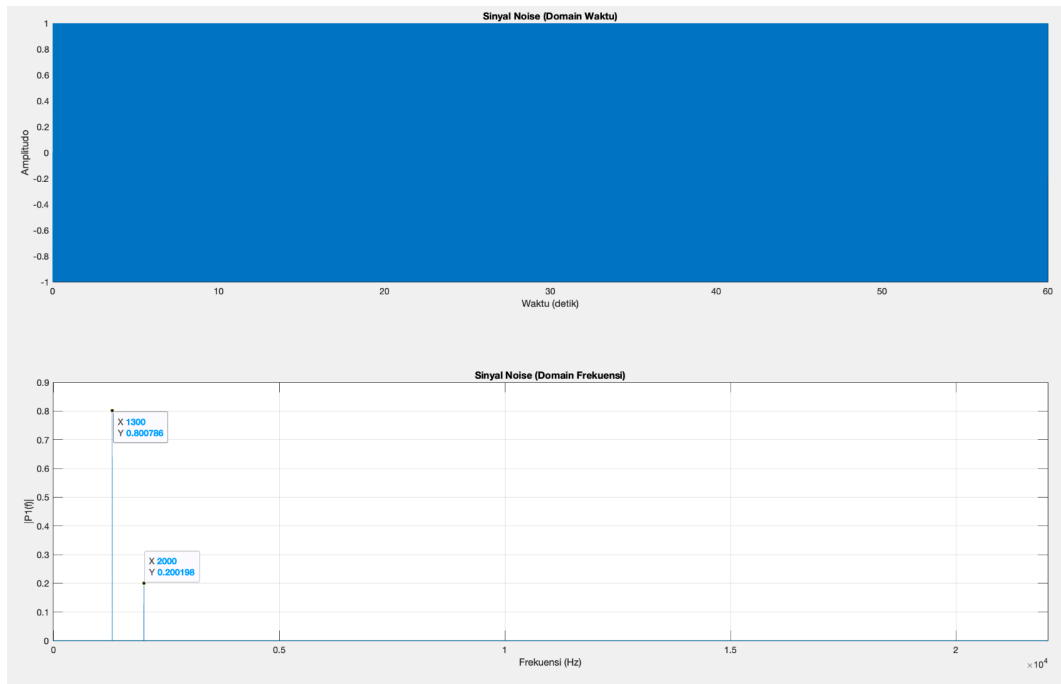


Gambar Ilustrasi AI untuk soal 3

a. Code audioOut.

Jawab:

Pertama, dilakukan analisis karakteristik sinyal *noise* pada domain frekuensi untuk menentukan frekuensi *cut-off*:



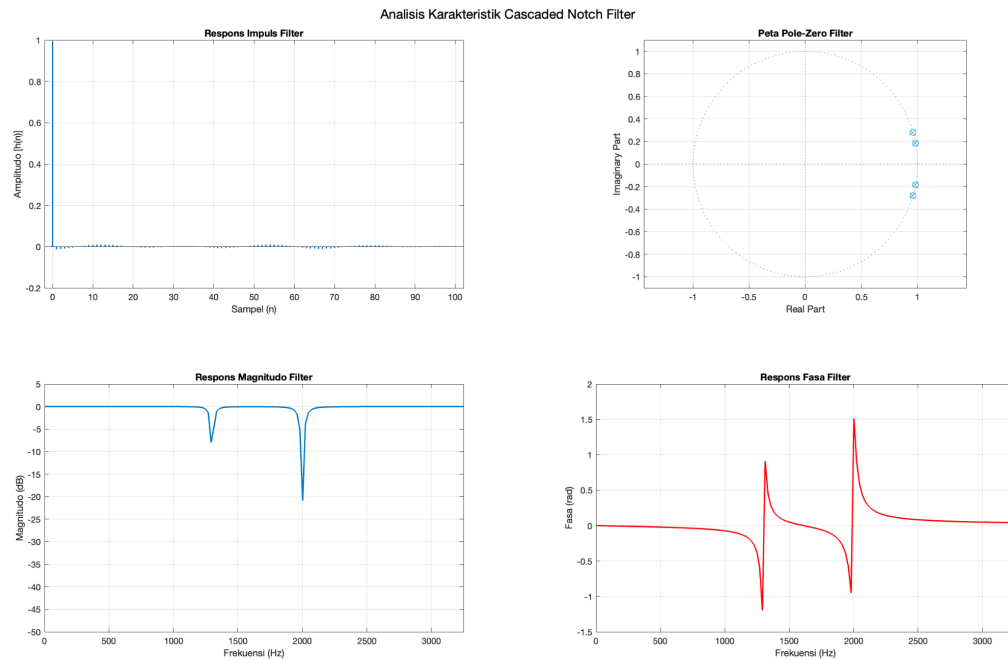
Gambar 47 - Sinyal Noise di Domain Frekuensi

Berdasarkan hasil analisis di domain frekuensi, ditemukan adanya puncak pada frekuensi 1300 Hz dan 2000 Hz. Kemudian, dilakukan implementasi *cascaded notch filter* pada kode MATLAB sebagai berikut,

```
Q = 35;
[b1, a1] = iirnotch(1300/(Fs/2), (1300/(Fs/2))/Q);
[b2, a2] = iirnotch(2000/(Fs/2), (2000/(Fs/2))/Q);

z1 = zeros(max(length(a1), length(b1))-1, 1);
z2 = zeros(max(length(a2), length(b2))-1, 1);

[temp, z1] = filter(b1, a1, audioIn, z1);
[audioOut, z2] = filter(b2, a2, temp, z2);
```

Gambar 48 - Analisis Karakteristik Cascaded Notch Filter

Filter tersebut diimplementasikan dalam kode audioOut untuk menghasilkan suara ucapan manusia dengan *noise* seperti sebagai berikut,

```
%% Setting
Fs = 44100;
frameSize = 1024;
mic = audioDeviceReader('SampleRate', Fs, 'SamplesPerFrame', frameSize);
speaker = audioDeviceWriter('SampleRate', Fs);
fileWriterInput =
dsp.AudioFileWriter('mySpeechInput.wav', 'FileFormat', 'WAV', 'SampleRate', Fs);
fileWriterOutput =
dsp.AudioFileWriter('mySpeechOutput.wav', 'FileFormat', 'WAV', 'SampleRate', Fs);
Q = 35;
[b1, a1] = iirnotch(1300/(Fs/2), (1300/(Fs/2))/Q);
[b2, a2] = iirnotch(2000/(Fs/2), (2000/(Fs/2))/Q);
z1 = zeros(max(length(a1), length(b1))-1, 1);
z2 = zeros(max(length(a2), length(b2))-1, 1);
%% Proses
disp('Silakan bicara sekarang...');
hFig = figure('Name', 'Tekan tombol apapun untuk berhenti', 'KeyPressFcn', @(src,event)
set(src, 'UserData', 1));
set(hFig, 'UserData', 0);
while ishandle(hFig) && get(hFig, 'UserData') == 0
    audioIn = mic();

    [temp, z1] = filter(b1, a1, audioIn, z1);
    [audioOut, z2] = filter(b2, a2, temp, z2);

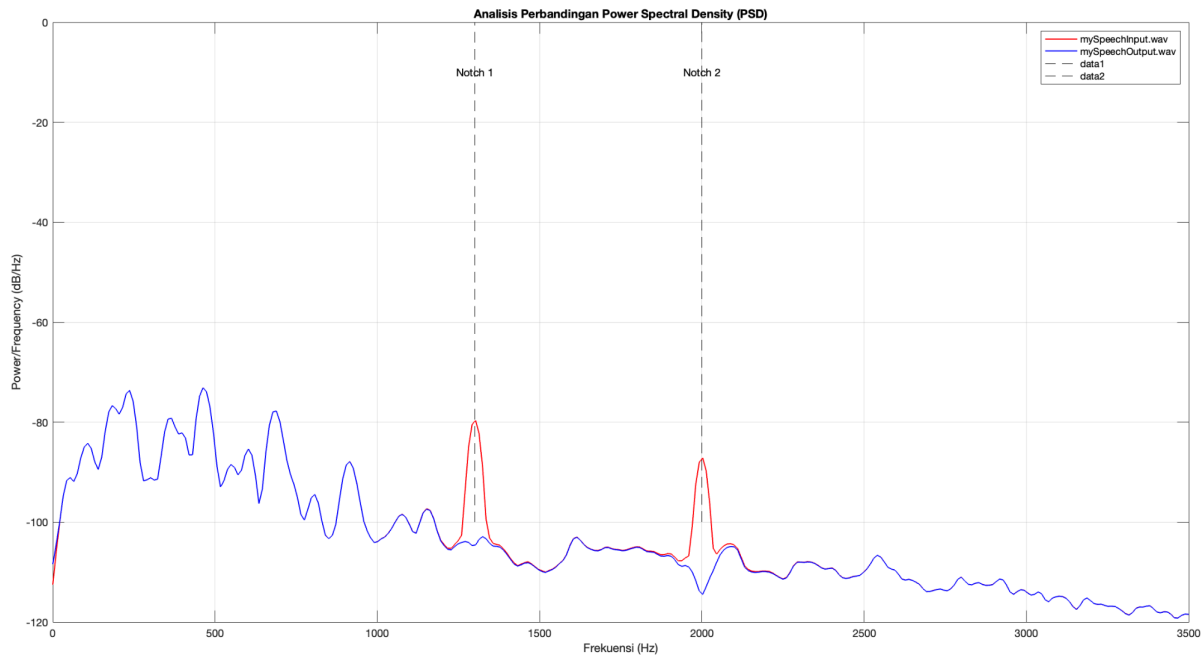
    speaker(audioOut);
    fileWriterInput(audioIn);
    fileWriterOutput(audioOut);
    drawnow;
end
disp('Selesai');
%% Pekerjaan rutin: release memori
```

```

release(mic)
release(speaker)
release(fileWriterInput)
release(fileWriterOutput)
if ishandle(hFig)
    close(hFig);
end

```

Gambar 49 - Kode Matlab Soal 3a



Gambar 50 - Analisis Perbandingan PSD Pada mySpeechInput.wav dan mySpeechOutput.wav

Grafik respons magnitudo filter menunjukkan *dip* tajam pada kedua frekuensi noise, dengan penurunan magnitudo mencapai -20 hingga -50 dB. Hal ini menunjukkan bahwa filter memotong *noise* tanpa merusak kualitas suara manusia di sekitarnya. Pada diagram pole-zero, *zeros* terdapat tepat pada lingkaran satuan pada sudut frekuensi *noise*. Kedekatan *zeros* ini menyebabkan respons sistem bernilai sangat kecil dalam skala dB pada frekuensi interferensi tersebut. Pada grafik respons impuls filter, terlihat sinyal *output* menuju nol setelah diberikan impuls input, sehingga filter bersifat stabil dan tidak menghasilkan osilasi selama pemrosesan *real-time*. Pada grafik respon fasa filter, terlihat adanya pergeseran fasa yang tajam pada frekuensi *notch*.

Setelah implementasi filter, dilakukan analisis *Power Spectral Density* (PSD) yang membandingkan mySpeechInput.wav dengan mySpeechOutput.wav. Pada grafik PSD sinyal input, terlihat daya yang sangat tinggi pada frekuensi 1300 Hz dan 2000 Hz. Setelah diimplementasikan filter, daya pada kedua frekuensi tersebut berkurang secara drastis yaitu lebih dari 20 dB/Hz. Meskipun terjadi pengurangan daya pada frekuensi *noise*, kurva PSD sinyal output menunjukkan daya pada frekuensi vokal lainnya tetap dijaga.

b. Code sendiri implementasi Direct Form tipe I untuk mendapatkan audioOut.

Jawab:

Berikut adalah kode Matlab implementasi *Direct Form* tipe I untuk mendapatkan audioOut,

```

%% Setting

```

```

Fs = 44100;
frameSize = 1024;
mic = audioDeviceReader('SampleRate', Fs, 'SamplesPerFrame', frameSize);
speaker = audioDeviceWriter('SampleRate', Fs);
fileWriterInput =
dsp.AudioFileWriter('mySpeechInput.wav', 'FileFormat', 'WAV', 'SampleRate', Fs);
fileWriterOutput =
dsp.AudioFileWriter('mySpeechOutput.wav', 'FileFormat', 'WAV', 'SampleRate', Fs);
Q = 35;
[b1, a1] = iirnotch(1300/(Fs/2), (1300/(Fs/2))/Q);
[b2, a2] = iirnotch(2000/(Fs/2), (2000/(Fs/2))/Q);
z1 = zeros(max(length(a1), length(b1))-1, 1);
z2 = zeros(max(length(a2), length(b2))-1, 1);
%% Proses
disp('Silakan bicara sekarang...');
hFig = figure('Name', 'Tekan tombol apapun untuk berhenti', 'KeyPressFcn', @(src,event)
set(src, 'UserData', 1));
set(hFig, 'UserData', 0);
s1 = zeros(4, 1);
s2 = zeros(4, 1);
while ishandle(hFig) && get(hFig, 'UserData') == 0
    audioIn = mic();
    temp = zeros(size(audioIn));
    audioOut = zeros(size(audioIn));

    for n = 1:length(audioIn)
        curr_x = audioIn(n);
        curr_y = b1(1)*curr_x + b1(2)*s1(1) + b1(3)*s1(2) - a1(2)*s1(3) - a1(3)*s1(4);
        s1 = [curr_x; s1(1); curr_y; s1(3)];
        temp(n) = curr_y;
    end

    for n = 1:length(temp)
        curr_x = temp(n);
        curr_y = b2(1)*curr_x + b2(2)*s2(1) + b2(3)*s2(2) - a2(2)*s2(3) - a2(3)*s2(4);
        s2 = [curr_x; s2(1); curr_y; s2(3)];
        audioOut(n) = curr_y;
    end

    speaker(audioOut);
    fileWriterInput(audioIn);
    fileWriterOutput(audioOut);
    drawnow;
end
disp('Selesai');
%% Pekerjaan rutin: release memori
release(mic)
release(speaker)
release(fileWriterInput)
release(fileWriterOutput)
if ishandle(hFig)
    close(hFig);
end

```

Gambar 51 - Kode Matlab Soal 3b

Pengimplementasian Direct Form I pada kode menyatakan filter IIR secara eksplisit untuk memisahkan bagian feedforward dan feedback. yang dimana setiap sampel dihitung dengan persamaan beda berdasarkan nilai input saat ini dan 2 input sebelumnya, serta menggunakan 2 sampel output

sebelumnya. hal ini tergambarkan pada vektor state (s1) yang memiliki ukuran empat elemen, dimana 2 elemen pertama menyimpan riwayat input dan 2 elemen terakhir menyimpan riwayat output. struktur ini membutuhkan total empat unit delay untuk filter orde-2, sehingga tidak optimal dalam penggunaan memori tetapi juga memiliki keunggulan dalam stabilitas numerik. karena tidak ada penguatan internal yang signifikan pada state, DF-I kurang rentan terhadap perhitungan error kuantisasi, terutama pada filter dengan pole yang dekat dengan lingkaran satuan. seperti yang ditunjukkan pada notch filter dengan $Q = 35$ yang dirancang untuk freq 1300 Hz dan 2000 Hz. DF-I memberikan respons yang bagus dan mudah diverifikasi secara konseptual meskipun lebih banyak penggunaan memori state pada saat audio real timenya.

c. Code sendiri implementasi Direct Form tipe II untuk mendapatkan audioOut.

Jawab:

Berikut adalah kode Matlab implementasi *Direct Form* tipe II untuk mendapatkan audioOut,

```
%% Setting
Fs = 44100;
frameSize = 1024;
mic = audioDeviceReader('SampleRate', Fs, 'SamplesPerFrame', frameSize);
speaker = audioDeviceWriter('SampleRate', Fs);
fileWriterInput =
dsp.AudioFileWriter('mySpeechInput.wav', 'FileFormat', 'WAV', 'SampleRate', Fs);
fileWriterOutput =
dsp.AudioFileWriter('mySpeechOutput.wav', 'FileFormat', 'WAV', 'SampleRate', Fs);
% Desain Filter Notch untuk 1300 Hz dan 2000 Hz
Q = 35;
[b1, a1] = iirnotch(1300/(Fs/2), (1300/(Fs/2))/Q);
[b2, a2] = iirnotch(2000/(Fs/2), (2000/(Fs/2))/Q);
% Inisialisasi State
z1 = zeros(max(length(a1), length(b1))-1, 1);
z2 = zeros(max(length(a2), length(b2))-1, 1);
%% Proses
disp('Silakan bicara sekarang...');
hFig = figure('Name', 'Tekan tombol apapun untuk berhenti', 'KeyPressFcn', @(src,event)
set(src, 'UserData', 1));
set(hFig, 'UserData', 0);
s1 = zeros(2, 1);
s2 = zeros(2, 1);
while ishandle(hFig) && get(hFig, 'UserData') == 0
    audioIn = mic();
    temp = zeros(size(audioIn));
    audioOut = zeros(size(audioIn));

    for n = 1:length(audioIn)
        w_curr = audioIn(n) - a1(2)*s1(1) - a1(3)*s1(2);
        curr_y = b1(1)*w_curr + b1(2)*s1(1) + b1(3)*s1(2);
        s1 = [w_curr; s1(1)];
        temp(n) = curr_y;
    end

    for n = 1:length(temp)
        w_curr = temp(n) - a2(2)*s2(1) - a2(3)*s2(2);
        curr_y = b2(1)*w_curr + b2(2)*s2(1) + b2(3)*s2(2);
        s2 = [w_curr; s2(1)];
        audioOut(n) = curr_y;
    end
end
```

```

    speaker(audioOut);
    fileWriterInput(audioIn);
    fileWriterOutput(audioOut);
    drawnow;
end
disp('Selesai');
%% Pekerjaan rutin: release memori
release(mic)
release(speaker)
release(fileWriterInput)
release(fileWriterOutput)
if ishandle(hFig)
    close(hFig);
end

```

Gambar 52 - Kode Matlab Soal 3c

Pengimplementasian Direct Form II pada kode menggunakan 1 rangkaian delay minimum yang dapat menyimpan sinyal internal $w[n]$ yang merupakan hasil dari bagian feedback sebelum dikalikan dengan koef feedforward. Vektor state (s1) pada DF-II hanya berukuran 2 elemen yang merepresentasikan $w[n-1]$ dan $w[n-2]$, yang digunakan dalam perhitungan feedback dan feedforward. Struktur ini dikenal dengan bentuk kanonik, karena hanya memerlukan $\max(N, M) = 2$ unit delay untuk filter order 2 sehingga dapat menghemat memori. tetapi efisiensi ini memiliki risiko ketika pole filter berada sangat dekat dengan lingkaran satuan seperti yang terlihat pada notch filter dengan $Q=35$, sinyal $w[n]$ dapat mengalami penguatan yang besar sehingga rentan terhadap saturasi atau error pada numerik sistem dengan presisi yang terbatas. Meskipun demikian DF-II tetap menghasilkan output yang akurat dengan kompleksitas state yang lebih rendah.

Pelajari *built-in functions* berikut:

audioDeviceReader, audioDeviceWriter

Nilai: 30

a	b	c
20	5	5

Referensi

- [1] [Online]. Available: https://docs.google.com/spreadsheets/d/1boJNvGXAMCT0Z29DT2QAGai_wIk7-bbTRIJ3fabOqLk/edit?usp=sharing
- [2] [Online]. Available: <https://www.mathworks.com/help/signal/digital-and-analog-filters.html>
- [3] M. Liebing, A. S. Imani, F. P. Atmadja, A. P. Gitasya, N. Awaliyah. (2025). *PSB_TugasBesar_Kelompok4*. GitHub.
https://github.com/BingBongCoder/PSB_TugasBesar_Kelompok4.git